West Visayas State University

COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY

La Paz, Iloilo City, Philippines

NeoDev: A Visual Web Dev IDE and Learning Tool utilizing Graph-Based Classification and Front-End Framework Conversion via GraphSAGE and Rules-Based System Code Transpilation

An Undergraduate Thesis

Presented to the Faculty of the

College of Information and Communications Technology

West Visayas State University

La Paz, Iloilo City

In Partial Fulfillment

of the Requirements for the Degree

Bachelor of Science in Computer Science

by

Rei Ebenezer G. Duhina

Marc Joshua H. Escueta

John Albert T. Claveria

Joshua Prinze C. Calibjo

Prince Alexander D. Malatuba

June 2026

West Visayas State University

COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY

La Paz, Iloilo City, Philippines

Approval Sheet

NeoDev: A Visual Web Dev IDE and Learning Tool utilizing Graph-Based Classification and Front-End Framework Conversion via GraphSAGE and Rules-Based System Code Transpilation

An Undergraduate Thesis for the Degree

Bachelor of Science in Computer Science

by

Rei Ebenezer G. Duhina

Marc Joshua H. Escueta

John Albert T. Claveria

Joshua Prinze C. Calibjo

Prince Alexander D. Malatuba

Dr. Frank I. Elijorde Panel	Dr. Arnel N. Secondes Panel
Dr. Ma. Luche P. Sabayle Panel	Mr. John Christopher Mateo Panel

Professor Mark Joseph J. Solidarios
Adviser

Concurred:

Ralph Voltaire Dayot Dept./Div. Chair, CS	Dr. Ma. Beth S. Concepcion Dean

June 2026

Acknowledgment

The researchers would like to express their deepest appreciation to the following persons, who in one way or another have made this work possible:

We would like to thank our adviser, Professor Mark Joseph J. Solidarios for helping us with the development and creation of this thesis. His advice and direction have been invaluable in shaping the final product of this thesis

We would like to thank all those who have supported us, our classmates, our parents, who made sure that we could focus on our work, even when things got hectic.

Above all else, we would like to thank God for giving us his wisdom and his providence in allowing us to successfully complete this project.

Rei Ebenezer G. Duhina

Marc Joshua H. Escueta

John Albert T. Claveria

Joshua Prinze C. Calibjo

Prince Alexander D. Malatuba

June 2026

R.E.G. Duhina; M.J.H. Escueta; J.A.T. Claveria; J.P.C. Calibjo; P.A.D. Malatuba, "NeoDev: A Visual Web Dev IDE and Learning Tool utilizing Graph-Based Classification and Front-End Framework Conversion via GraphSAGE and Rules-Based System Code Transpilation" Unpublished Undergraduate Thesis, Computer Science, College of Information and Communications Technology, West Visayas State University, Iloilo City, Philippines, 2026.

Abstract

Web development in the entirety of its existence requires mastery of HTML, CSS, and JavaScript, which introduces a steep learning curve. GUI-based site builders and prototyping tools have been developed to abstract away the inherent complexity of these languages, at the expense of the learning process. The researchers created NeoDev to abstract away the difficulty of learning code syntax while keeping the learning structure intact by providing a drag and drop-based development environment for creating websites. A hierarchical GraphSAGE classifier has been developed to give additional insights for developers in NeoDev. The model predicts which real-world component best matches the collection of blocks the user develops. The model has been trained on a labeled dataset of DOM Element collections, and tested based on performance metrics compared to CNN and GCN models. GraphSAGE performed excellently in both normal and semantically bleached datasets,

as well as during training, however the shallowness of the graphs allowed GCN and CNNs to achieve parity with the model. Overall, GraphSAGE is capable of classifying DOM Components.

Keywords: Web Development, GraphSAGE, classifier, IDE, GUI, Drag-and-drop, component, DOM Element

Table of Contents

List of Figures

List of Tables

List of Appendices

CHAPTER 1 INTRODUCTION TO THE STUDY

Background of the Study and Conceptual Framework

Web development has undergone significant advancements with the continuous emergence of new technologies, frameworks, and methodologies. However, the fundamental technologies required to learn and practice web development have remained unchanged since their inception [1]. A solid understanding of HyperText Markup Language (HTML) for structuring content, Cascading Style Sheets (CSS) for presentation, and JavaScript for dynamic behavior has consistently formed the foundation of web development [2]. Despite the evolution of tools and techniques, these three languages remain indispensable to mastering the discipline [3].

Learning these core technologies presents a steep learning curve, as each language follows a distinct coding paradigm. HTML is a markup language, CSS follows a cascading styling model, and JavaScript is a programming language [4]. As a result, beginners must navigate multiple paradigms,

each with unique syntax and rules, thus making the learning process inherently complex.

To address this challenge, various prototyping tools have been introduced, offering an alternative approach to web development. Platforms such as WordPress, Figma, and Webflow abstract away much of the technical complexity by allowing developers to build websites through graphical user interfaces rather than direct coding [5]. These tools provide a smoother learning curve, enabling users to focus more on design thinking and logical structuring rather than the intricacies of coding. However, while these platforms allow for rapid prototyping and increased accessibility, they often come at the cost of reduced control and flexibility, limiting developers' ability to implement custom functionality and optimize performance at a deeper level [6]. Additionally, it also prevents developers from learning the concepts of web development as these platforms stray from its fundamentals.

A comparable approach to simplifying programming concepts can be observed in Scratch, a graphical programming

language developed by researchers at the Massachusetts Institute of Technology (MIT). Scratch employs a block-based interface to introduce young learners to fundamental programming principles without requiring textual coding [7]. By eliminating syntactic barriers, Scratch fosters logical reasoning and problem-solving skills, demonstrating that intuitive design tools can serve as effective educational platforms for novice developers.

Despite the increasing availability of abstraction tools, a comprehensive understanding of HTML, CSS, and JavaScript remains paramount in modern websites. However, the success of graphical programming environments such as Scratch suggests that a similar abstraction-based methodology could be developed for web development while preserving the essential separation of concerns inherent in HTML, CSS, and JavaScript. By implementing a block-based system where structural elements (HTML), styling components (CSS), and logical operations (JavaScript) are distinctly represented, learners could engage with web development concepts more intuitively without sacrificing the

foundational principles that underpin modern web technologies.

GraphSAGE is a powerful framework for learning node representations or embeddings in large graphs. Previously, traditional graph embedding methods utilized transductive training, which required them to retrain on new or unseen nodes. GraphSAGE sidesteps this by employing inductive methods, which enable the GNN to learn functions that allow it to create embeddings by sampling and aggregating data from local neighborhoods. Additionally, GraphSAGE is designed for large and irregular graphs that are capable of evolving dynamically, such as the structures and frameworks of Websites that are represented as Document Object Models (DOMs). These DOMs often appear as trees, with both hierarchical relationships (parent-child-grandchild) and highly irregular and varying structures. As such, the Graphs Generated by these DOMs tend to be highly varied in depth, node types, and any embedded CSS. Furthermore, these DOM trees are rich in features and result in deep trees that may end up being discarded as noise in other GNNs. GraphSAGE, on the other hand, excels in handling these structures. Given

that these structures also repeat themselves across the web, GraphSAGE's ability to learn functions rather than direct embeddings allows it to classify similar graphs and takes into account any slight changes in structure.

The development of modern web applications has transcended simple script authoring, evolving into the complex management of interdependent components, localized styles, and dynamic data structures. Current architectural paradigms often treat these modules as disparate entities, which frequently results in significant technical debt, difficulty in maintaining design consistency, and barriers to efficient code reuse. As projects scale, the manual effort required to adapt existing UI elements to new environments or design systems becomes a primary bottleneck in the software development life cycle.

To address these challenges, this study introduces NeoDev, an intelligent, inductive development environment engineered for the automated generation and management of reusable web components. Unlike traditional Integrated Development Environments (IDEs) that rely on static

templates or manual definitions, the proposed system utilizes an inductive learning approach. This architecture is designed to sample and aggregate structural and stylistic features from established project libraries, facilitating the generation of new components that are natively compatible with a specific design system.

Central to the NeoDev environment is the implementation of GraphSAGE, a graph-based framework that enables the system to learn a generative function. By representing the Document Object Model (DOM) as a graph, the system can aggregate attributes from existing page elements—such as navigation bars, content headers, and action buttons—to produce coherent, responsive, and optimized new layouts.

The primary advantage of this inductive capability is its ability to generalize to unseen designs. By training on a localized set of corporate or project-specific websites, NeoDev can autonomously generate new pages that maintain structural and brand integrity without the necessity of manual reconstruction. This approach not only streamlines

the development workflow but also ensures a standardized level of visual and functional consistency across heterogeneous web projects.

Objectives of the Study

This study focuses on building a web development learning tool specializing in specific front-end web frameworks that utilizes a GraphSAGE neural network classifier for optimization in tandem with a rules-based system for transpilation to frontend framework code to streamline the learning process of how websites work.

Specifically, it aims to:

1. Develop a visual-focused web development, software as a service (SaaS) learning tool called NeoDev with block-based drag-and-drop functionality similar to Scratch.
2. Develop a GraphSAGE neural network classifier to optimize code and provide insights. Train this classifier on custom intermediary code datasets created through the prototyping platform and real-world web-scraped sites.
3. Create a code transpiler that converts code to a specific frontend framework chosen by the user that Rules-based systems on the compilation step, and

ensures each system is tailored to its target frontend framework.

4. Evaluate the accuracy and precision of the GraphSAGE classifier, and the transpilation accuracy and speed of the compiler in comparison to similar platforms such as Builder.io's Figma-to-code plugin.
5. Evaluate the performance, user experience, and accessibility of the system based on the ISO 9241 series and the W3C Web Content Accessibility Guidelines (WCAG) 2.0. via user feedback and testing using questionnaires.

Significance of the Study

The Results of this Study will be beneficial to the following:

1. **ICT Students and Independent Learners** – NeoDev serves as a platform to learn web development through a GUI-based interface, allowing room for logical flow without the complexities of syntactic sugar.

2. **Teachers and Educators** – NeoDev serves as an alternative learning environment for their students in web development, particularly for those who find traditional methods of instruction impractical.

3. **Scholarly Institutions** – Schools and Universities can use this tool to develop new curricula based on Visual Web Development.

4. **Small Development Teams —** They can use this to assist the onboarding process of new developers and engineers, who must learn the company's proprietary libraries and frameworks before they can start working.

5. **Future Researchers** – They can use this study as a reference to their research on visual-based coding, and/or take inspiration from this study in their future research.

Delimitations of the Study

This study focuses on the structured representation of HTML component structures for classification using Graph Neural Networks (GNNs). To ensure the feasibility and clarity of the dataset, the following delimitations have been established:

Exclusion of JavaScript in Model Training

JavaScript will not be included as a parameter in the dataset. The primary reason for this exclusion is the inherent difficulty of quantifying programming logic consistently within the dataset's scope. Unlike HTML and CSS, JavaScript involves dynamic interactions and dependencies that are challenging to standardize for classification purposes.

Use of Frontend Code Only

The compilation step will only produce frontend code. Backend code introduces additional layers of complexity, including data processing, authentication, and server-side logic, which are outside the scope of this study.

Furthermore, compiling backend code would require significantly more computational resources and implementation effort, making it impractical for this research.

Definition of Terms

For better understanding, the following terms are defined:

1 NeoDev (n.)

a. Conceptual Definition - a combination of two words: *neo*, literally meaning *new*, and *dev*, a contraction of the words *developer* or *development*.

b. Operational definition - refers to the name of the learning tool and Web Development IDE being developed as part of this study.

2 Tag Blocks (n.)

a. Conceptual Definition - a collection of HTML elements that form the structure of a webpage.

b. Operational definition - a draggable block that represents an HTML tag. It can contain inherent attributes depending on the type, and style blocks can be attached to it.

3 Style blocks (n.)

a. Conceptual Definition - a section of code typically written in CSS that determines the styling of an HTML element.

b. Operational Definition - a draggable block that represents a CSS style declaration, containing a property name and a property value.

4 Frame (n.)

a. Conceptual Definition - a container or collection of elements.

b. Operational Definition - groups tag and style blocks together in a collection. Allows for encapsulation and componentization in NeoDev.

CHAPTER 2 REVIEW OF RELATED STUDIES

Review of Existing and Related Studies

Current Systems:

Existing Visual Programming Languages:

Visual Programming languages are languages that utilize two-dimensional graphical elements representing programming constructs and rules in a visual manner that allows non-programmers to create, extend, and customize applications [8]. Based on the existing taxonomies of Myers [9], and Burnett and Baker [10], there are currently four different types of visual programming languages: form-based, block-based, diagram-based, and icon-based. Block-based programming Languages allow users to drag and drop blocks that represent elements of a program into the development environment from a set of predefined commands. These blocks are then strung together to create a program. This type of VPL prevents syntax errors and allows the user to focus more on the concepts rather than the act of programming itself. Examples of this include Scratch and App Inventor, which have made programming much more accessible to the end user.

One of these, Scratch, is a high-level programming language that uses visual building blocks to construct programs. Developed in 2004, by the Lifelong Kindergarten Group in collaboration with Yasmin Kafai at UCLA, It is an emerging technology that can be applied to interactive game design, animation, and multimedia projects in the classroom with a target audience of ages 8-16 [11]. Its trial-and-error design allows students to play with the code, and modify and change parameters as needed, all while coding without the need to write any algorithms. [12]. It's free to use and is available in over 70 languages. This accessibility, combined with its user base, makes it one of the best tools to start teaching students from a young age [11, 12]

Another application that utilizes a block-based approach to visual programming is MIT's App Inventor. MIT App Inventor is an online application development platform that uses a web-based drag-and-drop "What you see is what you get" (WYSIWYG) visual editor for building Android and IOS Mobile Apps. [13, 14]. It uses a block-based programming language built on Google Blockly and inspired by languages such as StarLogo and Scratch and allows users to build any

app they want with ease. [13]. Like with Scratch, the App Inventor's intuitive block-based programming and incremental development capabilities allow the developer to focus more on the app's logic rather than syntax, and as a result, it has gained a steady following of users, reaching two million registered users with 18 months, and has over 40,000 weekly users from all over the world. [14]

Existing No-Code/Low-Code Web-Based Development Tools

The concept of low-code development originated in earlier Rapid Application Development tools aiming to expedite software development visually and reusable components using model-driven logic and pre-built templates, reducing the need for coding. This approach enables non-developers and also developers to create applications efficiently [15]. Most Low-Code Development Platforms are offered via Cloud through a Platform-as-a-Service (PaaS) model and enable the creation and deployment of fully functional software applications. As the name suggests, they utilize graphical user interfaces and visual abstractions requiring minimal or no procedural code, and thus, allow end-users with no particular programming

background to contribute to software development, without sacrificing the productivity of professional developers. [16]

While not strictly considered a full-fledged low-code development platform, Figma is one such application. It is a collaborative design tool that allows designers and developers to create interfaces with minimal coding [17]. It is a powerful platform for designers to apply user-centered design practices in the development of web-based learning media. With Figma, designers can create interactive prototypes, conduct user testing, and refine the interface to deliver a seamless and engaging learning experience for the target audience [18]. As a real-time collaborative UI design tool, people who work on the same project can view and edit the design at the same time, as well as make comments on the project removing a lot of unnecessary steps in the process Furthermore, it allows you to share the most up-to-date version of the project via the cloud, removing the hassle of transferring data manually [19]. Additionally, Figma also has several plugins and additional features such as FlutterFlow and Builder.io, among others,

that can be used to turn Figma's visual prototype into working code.

On the other hand, Ycode is a relatively new low-code/no-code web development platform that allows users to build websites visually with a drag-and-drop editor together with built-in database management and API integration features. Mostly suitable for startups and businesses looking to launch applications quickly without hiring a full development team [20]. Intended by its creators to serve as a tool that would allow anyone to create web apps without code, it has a built-in design editor that allows users to create websites via dragging and dropping of elements, and then publish it to a Ycode domain, or a custom domain for a monthly subscription fee [21].

Furthermore, it is seen in this study [22] that in a low-code approach to web development education, students performed much better on ICT skill development activities than on previous ICT disciplines, and noted that there was a 96% increase in motivation using this technology.

Existing Web Development Learning Tools

Most existing Web Development Learning tools are focused on learning the syntax of the code as opposed to the logic. Sites such as W3Schools.com [22] or the Odin Project [23] focus more on programming languages as opposed to the visual design and structure of the website such as in No-Code/Low-Code Websites. Additionally, there are a few other applications that also focus on learning web development. Encode is one such mobile learning application developed by Upsew Pty. Ltd. It provides courses for learning programming languages like Python, JavaScript, HTML, and CSS structured into segmented lessons with coding challenges, allowing their users to learn at their own pace, even offline [24]. Furthermore, certain frameworks and languages offer web design playgrounds. These playgrounds are an online platform designed for beginners in HTML and CSS. It provides real-time visual feedback as the user codes, making it an effective learning tool for front-end development as it generates hands-on experience [26]. Examples of Web Playgrounds include the aforementioned W3Schools.com, Coursera, FreeCodeCamp, and many more.

Related Systems or Solutions

Existing Prototype-to-Framework Solutions

Prototype-to-Framework is a term that refers to the idea of taking an existing visual prototype developed in Figma and transforming it into working code. Currently, there are a number of existing solutions that can convert a visual prototype into code. One particular example is Builder.io [27]. Builder.io is a visual development platform that allows the user to create fully functional websites by designing them, either in the app's built-in web designer or by importing a design from Figma via a plugin. [28]. Builder.io advertises itself like similar CMSs like Squarespace or Ycode, however, Builder.io sets itself apart by dynamically coding the layout as opposed to hard-coding the layout in a traditional CMS, as well as integration of your codebase. Another similar application is Flutterflow, which is a visual editor for Flutter-based applications, but unlike this study and builder.io, Flutterflow is restricted to a different web development language (Flutter/Dart) whereas builder.io can generate code for several different frameworks.

Related Studies

GraphSAGE and GNNs

Several studies have explored the capabilities of GNNs in various domains. Graph Convolutional Networks (GCNs) were introduced, leveraging spectral graph theory to define convolutional operations on graphs. GCNs have since been widely adopted for tasks like semi-supervised node classification and link prediction.[29]

GraphSAGE was introduced as an inductive learning framework designed to scale to large graphs by sampling and aggregating information from neighboring nodes. The study demonstrated that GraphSAGE could efficiently generate embeddings for unseen nodes, making it particularly useful in real-world applications like recommendation systems and biological network analysis.[30] Graph Isomorphism Networks (GIN) were proposed, highlighting the expressive power of different GNN architectures. The findings contributed to a better understanding of how GNNs can learn effective graph representations. [31] GNNs were applied in large-scale applications such as Pinterest's recommendation system, showcasing their ability to enhance content discovery. [32]

More recent studies have combined GraphSAGE with rule-based systems for specialized applications. For example, researchers have explored using GraphSAGE in educational tools, such as web development learning environments, where prototype-to-framework conversion can be enhanced through graph-based learning approaches.

GNNs leverage message-passing mechanisms to aggregate information from neighboring nodes and edges. The fundamental idea behind GNNs is that each node in a graph updates its representation by aggregating information from its local neighborhood iteratively. A general GNN framework consists of several steps. First, each node starts with an initial feature representation, such as word embeddings, user profiles, or molecular properties. Next, nodes aggregate features from their neighboring nodes using an aggregation function in a process known as neighborhood aggregation or message passing. After aggregation, the features undergo a non-linear transformation, such as a neural network layer, to update the node embeddings. Finally, after several iterations of message passing, the node embeddings capture both structural and feature-based information, which can be used for downstream tasks like

node classification, link prediction, or graph classification. Different variations of GNNs include Graph Convolutional Networks (GCNs), Graph Attention Networks (GATs), and GraphSAGE.

GraphSAGE is a scalable inductive learning framework for graphs. Unlike traditional GNNs that rely on full-batch training, which requires access to the entire graph during training, GraphSAGE efficiently learns node embeddings by sampling a fixed-size neighborhood instead of using all neighbors. This allows GraphSAGE to scale to large graphs. [29]

GraphSAGE incorporates several important techniques to enhance its efficiency. First, it employs a sampling strategy where instead of aggregating information from all neighbors, which can be computationally expensive in large graphs, GraphSAGE samples a fixed number of neighbors for each node. The sampled neighbors' features are then aggregated using various techniques, including mean aggregation, which computes the average of neighbors' features; LSTM aggregation, which uses an LSTM network to process neighbor features sequentially; and pooling aggregation, which applies a max-pooling function over the

neighbors' feature representations. To ensure efficiency, GraphSAGE also utilizes weight sharing, where the aggregation function uses shared parameters across different nodes, enabling inductive learning and generalization to unseen nodes. Finally, the node's new embedding is generated by concatenating its own features with the aggregated neighborhood features and applying a non-linear transformation.

GraphSAGE offers several key advantages. One of its main benefits is inductive learning, meaning that, unlike transductive models like GCN, GraphSAGE generalizes to new, unseen nodes in a graph. Additionally, it is highly scalable; by using neighbor sampling, GraphSAGE reduces computational complexity and can scale to very large graphs. Moreover, GraphSAGE is flexible and supports different aggregation functions, allowing it to capture diverse types of graph structures and relationships.

GraphSAGE and GNNs have numerous applications across various domains. In social networks, they are used for predicting user interests, detecting fake accounts, and identifying communities. In recommendation systems, they enhance movie, product, and content recommendations by

leveraging graph embeddings. In biology and chemistry, they aid in protein-protein interactions, drug discovery, and molecular property prediction. In finance and fraud detection, they help identify fraudulent transactions and suspicious activities within financial networks. Furthermore, in natural language processing, GNNs contribute to knowledge graph completion, text classification, and question-answering systems.

World Wide Web and Web Development

Tim Berners-Lee, the inventor of the World Wide Web, envisioned a system where information could easily be shared and connected across the globe [1]. Now with his work laying the foundation and the web taking shape, it needs a visually appealing and user-friendly design to be interacted with. HTML and CSS, being the backbone of web page design, Duckett introduces these core technologies making them accessible to beginners while emphasizing the importance of clean and readable design [3].

Structure and design aside, modern web development evolved from interactivity to dynamic content as Feldman introduces Elm, a functional programming language designed

for front-end development focusing on web applications [5], and Resnik et al present *Scratch*, a programming language that simplifies coding through visual approach [7] .

CHAPTER 3 RESEARCH DESIGN AND METHODOLOGY

Description of the Proposed Study

The study employs a developmental research design to create a GUI-based web development learning platform and develop an AI classifier based on the GraphSAGE algorithm. The research is structured into two simultaneous parts: one for system development, and one for dataset gathering and model implementation, followed by an evaluation phase that assesses the model's accuracy.

A. NeoDev - platform design

The proposed platform implementation is a Software-as-a-Service (SaaS) web development tool that utilizes a block-based approach like Scratch, where users can develop and create a website by connecting blocks that represent an HTML Tag. Each user has its own local offline sandbox environment, where progress is automatically saved to the local storage on every action. Upon first visit, users will be welcomed to the title screen, as shown in Figure One, where they can open the development environment by clicking on "Open Playground."

Figure 1. The NeoDev title screen

B. Development Environment Design

The development environment consists of multiple panels, each with its own purpose. The playground panel is an open-world environment, containing groups of box elements known as blocks. The groups are defined as frames, and each frame has a label on the left side. Background colors on these labels represent state, where reddish-orange represents the template frame, purple represents the active frame, and the rest are colored yellow-orange. Zooming and panning on the playground is allowed through the use of

either keyboard shortcuts or mouse via the toolbar panel. The toolbar also allows for switching between move mode, allowing blocks and frames to be draggable, and pan and zoom mode, which allows navigation on the playground itself.

In the background, a snapshot of the development environment is converted into raw code and then tokenized before being fed to the GraphSAGE classifier hosted on render.com as a separate backend server. This tokenized code is then processed by the neural network and returns insights and warnings based on the user's design. Finally, once the user is satisfied with their website, they can use the code transpiler, which utilizes rule-based systems to convert their website to any frontend framework of their choice. This transpiler combines the user's design with optimizations from GraphSAGE and converts it to usable and industry-ready frontend framework code.

Methods and Proposed Enhancements

System Development

The system development phase follows an iterative process model. The first iteration focuses on completing system requirements and building the prototype, the second on backend integration and model communication, and the final on bug fixing and feature finalization.

Generating the Dataset and Ethical Considerations

Data gathering is done by extracting data from user-built NeoDev projects. If the number of projects proves insufficient, HTML component data from web-crawled websites will be used. NeoDev users are made aware in the application's End-User License Agreement (EULA) that projects made in the application will be used solely as training data for the GraphSAGE model, indicating the ethical considerations and privacy measures involved.

For initial testing, the Dataset was created using a specialized webscraper that allows users to manually copy a websites' HTML and CSS structure, paste it into the text area of the webscraper, and proceed to manually label the

HTML and CSS structure by selecting the correct labels based on its attributes and children. Once the entirety of the code has been parsed, and classified, the user can then save these labels to a json file, which is named labeled_data.json

Dataset Entry Structure

labeled_data.json is in JavaScript Object Notation (JSON) format, which is organized as a collection of independently annotated individual front-end components. Each entry is an isolated website component encoded in a structured Document Object Model (DOM) structure as detailed in Figure 2

At the highest level, each element of the JSON array corresponds to a single instance of a component, such as a Hero Section or a Card. Each entry contains two top-level fields. A “label” entry that identifies which component it is followed by “contents” a collection of DOM trees and subtrees that belong to the component. This structure is multi-rooted, and thus it does not necessarily have an HTML

Root Tag, and mainly contain independent top-level DOM nodes, which mirrors modern web design.

Each component's DOM within the content's list has the following fields: First, the "tag" or "element", which is the HTML tag name in lowercase. Second is "attributes", which is a dictionary encoding various HTML properties such as inline styles, class lists, image source, and so on. The third is "name", which is a descriptive name used to help in identifying specific nods. The fourth is children, which includes any nested DOM nodes within the same schema and allows for the representation of complex layouts with deep DOM trees.

Since real-world web components often feature multiple top-level containers, the dataset uses a multi-root representation, where a component's children field may contain several independent root nodes. This structure preserves the organic hierarchy of web UI code, enabling the GraphSAGE model to extract features from authentic component layouts rather than simplified or re-rooted trees. By maintaining this structural integrity, the dataset captures the semantic and structural properties of modern web design,

providing rich hierarchical data for the inductive learning process. During preprocessing, nodes become graph nodes, hierarchical relationships become graph edges, and attributes become numerical feature vectors. This allows the dataset to capture the semantic and structural properties of modern web design, allowing for rich hierarchical information to be fed towards GraphSAGE. Figure 2 provides an example of a dataset entry, showcasing the dropdown menu component and its component elements.

```
{
    "label": "DropdownMenu",
    "contents": [
        {
            "type": "div",
            "attributes": {},
            "name": "DropdownMenu",
            "children": [],
            "styles": [
                "color: #F632E7",
                "background-color: #82DEDD",
                "font-size: 23px",
                "padding: 8px",
                "margin: 11px",
                "border: 1px solid #D582C3"
            ]
        }
    ]
},
```

Figure 2. Sample Entry for Training Dataset with Label

Data Conversion and Preprocessing

Should the number of projects prove insufficient, as noted in Section 3.2.2, since the system relies on a JavaScript object for state management, the dataset's source data must be converted into a format compatible with GraphSAGE. This process includes multiple preprocessing steps and a labeling step to enhance the differentiation

between various components, ensuring more accurate classification.

For the dataset, the preprocessing stage can be broken down into seven stages.

1. Parsing Components and Enumerating DOM Trees

The initial pipeline commences by loading all entries from the compiled dataset. For each component, the "label" field is stored as a graph-level label while each entry in "contents" is treated as the root of another tree. A depth-first traversal (DFS) is formed over each tree, assigning a unique ID to each node, parent-child relationships, and sibling-ordering information. Three categories of information are gathered. First, there is the set of observed HTML-tags, which are normalized through the node_tag function and accumulated into a dictionary for one-hot encoding. Next, CSS Textual descriptors are extracted, concatenating class names, inline styles, and similar attributes into compact representations. These form a corpus for a TF-IDF vectorizer. Finally, a global StandardScaler is initialized in preparation for later

fitting. Two LabelEncoders, one for element and one for component are partially fit.

2. Structural Decomposition of Components

Each component is then decomposed into a list of element metadata via recursive transversal. For every node encountered, structural and contextual attributes are noted down, which includes DOM Depth, index among siblings, its parents, and child count. Tag Identity is preserved, and additional semantic indicators for interactive and media are added via predefined categories.

3. Subtree Statistic Computation,

Once all elements for a component have been identified, Depth First Aggregation is used to compute subtree-level statistics. These include the total number of descendants in each node, as well as the number of interactive and media descendants. To do this, the script constructs a children adjacency list that performs post-order traversal from each root. This enriches each node with contextual descriptors reflecting both its own role and the broader functional density of the subtree it dominates.

4. Construction of Graph Edges and Attributes

After the elements and subtrees have been computed, the preprocessor then constructs a Graph Topology extending beyond hierarchical and lateral relationships, including various edges. Three edge types are currently defined, we have parent and child, and child and parent edges, which are directional edges that handle the flow of information, for encoding vertical structure. There are also sibling edges for nodes with the same parent for horizontal order and layout grouping. Additionally, all tree roots in the "contents" field receive sibling edges relative to each other, as well as component-root should it be required for training, ensuring that the multi-root systems will become one graph. These are then converted into one-hot vectors for inclusion as "edge_attr" in the final graphs. If a component contains no edges, an empty but well formed graph structure is created.

5. Node Feature Vector Construction

Next, a comprehensive feature representation for every node is constructed. Each DOM Node is converted into a

feature vector, with the preprocessor extracting and encoding the “type” field, which contains the HTML tag name, via one-hot encoding. This becomes the primary categorical feature, and serves as a node-level label for hierarchical and element-only training. Then, CSS TF-IDF embeddings are extracted from the global model. If there are none, the features degrade gracefully to zero. Binary indicators determine whether a node is a leaf, interactive, or media element, as well as its place as the first or last child in its sibling group. Ancestor tag encodings provide contextual information by embedding the node’s parent and grandparent using separate one-hot vectors, allowing for vertical contextual awareness.

These are then condensed into a final matrix, which is then converted into a PyTorch Tensor

6. Fitting of Numeric Feature Scaler

Before constructing the final graph objects, a second pass is performed to obtain full numeric records with the subtree statistics. These are then aggregated to fit the

aforementioned StandardScaler, ensuring consistent normalization and stable optimization during training.

7. Conversion to PyTorch Geometric Graphs and data splitting

Finally, each component is converted into a PyTorch Geometric data object with x, being num_nodes and node_feature_dim, y being the element-level label vector (y), and the component-level label (component_y). There's also the edge_index and edge_attr for connectivity and one-hot attributes, along with the metadata fields describing node counts and original labels. All label encoders are fit using elements produced in the decomposition phase, ensuring alignment between encoded labels and the dataset. Malformed components are caught and logged, preventing pipeline interruption.

After graph construction, they are randomly shuffled and partitioned into training, validation, and test sets in a 70-15-15 ratio. Splits are formed at the graph level so that individual components form individual samples. Resulting graphs are serialized as three separate .pt files

as well as a metadata file for tag vocabulary, labels, CSS Vectorizer, and scaler.

Model Implementation and Training

For training and implementation of the model, a hierarchical training framework is used, and jointly optimizes classification on two levels: The node-level elements derived from the HTML tag of the "type" field, and the graph-level components. Additionally, the training pipeline integrates model construction, optimization, early stopping, and post-training evaluation, concluding with the production of a deployable model bundle.

The proposed model performs graph-level component classification by learning representations from lower-level DOM element nodes and their relationships. Each input sample is first represented as a graph $G=(V,E)$, where each node $v_i \in V$ corresponds to a DOM element and each edge $e_{ij} \in E$ represents a structural relationship such as parent-child, child-parent, or sibling adjacency. Node features encode structural, textual, and attribute-based information, while edge features encode the type or characteristics of the relationship between nodes.

At the core of the model is the GraphSAGEWithEdgeAttr layer, which extends the GraphSAGE message-passing framework by incorporating edge attributes into message construction. For each node i, the hidden representation at layer l+1 is computed as:

$$h_i^{(l+1)} = \text{Norm}\left(W_s h_i^{(l)} + \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \text{MLP}\left([h_j^{(l)} \parallel \phi(e_{ij})] \right) \right)$$

where hi(l) is the representation of node i at layer l, N(i) denotes the neighborhood of node i, Ws is the learnable self-feature transformation, ϕ(eij) is the learned embedding of the edge attribute between nodes i and j, and [·∥·] denotes concatenation. The function MLP(·) generates edge-aware messages from neighboring nodes, while Norm(·) denotes the optional normalization applied after aggregation.

Three such graph convolution layers are stacked in the HierarchicalGraphSAGE model to progressively refine node embeddings and capture higher-order structural dependencies. After the first and second layers, ReLU activation and dropout are applied to introduce nonlinearity and reduce

overfitting. Through this design, each final node embedding reflects not only the node's own properties but also the contextual influence of neighboring elements and their structural relations.

After the final convolution layer, the node embeddings are aggregated into a single graph-level representation using global mean pooling:

$$z_G = \frac{1}{|V_G|} \sum_{i \in V_G} h_i^{(L)}$$

where zG is the graph embedding for graph G, VG is the set of nodes in the graph, and L is the final graph convolution layer. This readout operation allows the model to generate a fixed-dimensional representation regardless of the number of nodes in the input graph.

The resulting graph embedding is then passed through a linear classification layer to produce the predicted component label:

$$\hat{y}_{\text{comp}} = \text{softmax}(W_c z_G + b_c)$$

where Wc and bc are the learnable parameters of the output layer, and y^comp is the predicted probability distribution over the component classes.

Although the final output of the model is a component-level prediction, the prediction is derived from the learned interactions of lower-level DOM elements. Thus, the proposed approach is hierarchical in the representational sense: higher-level component classification emerges from the composition, structure, and semantic interaction of lower-level element nodes.

GraphSAGE operates by updating node embeddings through aggregation of neighbor embeddings, for this implementation, edge attributes are incorporated into the message computation, allowing the network to distinguish structural relationships in the DOM tree. Each attribute is protected through the following process:

self.edge_proj = nn.Sequential(Linear(edge_dim → hidden)
ReLU, Linear(hidden → hidden))

This produces a learned edge embedding and allows the model to tell the differences between the three edge types. Following that, the GraphSAGE message function concatenates the following:

$$X_j \;||\; edge\_emb$$

Where X_j is the neighbor's node embedding and edge_emb is the embedded edge attribute. This concatenated vector is then passed through a small multilayer perceptron allowing relationally conditioned aggregation. Graphsage utilizes mean aggregation, and thus is represented by the following:

$$X_{new} = Self_{lin(x)} + mean(messages)$$

This addition ensures a stable gradient flow and supports deeper GNN Stacks. These consist of the following three stacked layers:

$$conv1 \rightarrow ReLU \rightarrow Dropout \rightarrow conv2 \rightarrow ReLU \rightarrow Dropout \rightarrow conv$$

This results in node embeddings becoming increasingly context-aware, capturing both local DOM semantics and the broader hierarchical organization.

After message passing, the model produces two different predictions, one for each level of classification. Node embeddings are passed through a linear transformation that creates the head_element. This performs semantic classification of individual DOM nodes. Meanwhile, graph embeddings are created using global mean pooling. This step aggregates the all node embeddings of the UI Component to create a single vector that captures the structure and visual semantics. For component level, the process is the same as element-level.

For the loss function, the training loop utilizes multi-task loss, combining element-level cross entropy and component-level cross entropy, along with optional label smoothing, improving generalization and preventing overconfidence in the model. The model was trained as a graph-level classifier using the component labels as the sole target variable.

Let C denote the number of component classes. The final training objective is the weighted cross-entropy loss:

$$L = L_{\text{comp}}$$

$$L = -\sum_{c=1}^{C} w_c y_c \log(\hat{y}_c)$$

where wc is the class weight assigned to class c, yc is the ground-truth indicator for class c, and y^c is the predicted probability for the same class. The class weights were computed from inverse class frequencies in the training set in order to reduce the effect of class imbalance and encourage more balanced learning across categories.

The training loop proper consists of the following steps:

1. Forward pass with dropout/edge dropout
2. Compute hierarchical loss
3. Backpropagation and gradient clipping
4. Compute F1 score for progress reporting
5. Validation evaluation

6. Monitor a combined metric (mean of element and component F1)

Training was performed using the Adam optimizer with a learning rate of 0.001 and a weight decay of 5×10−4. Mini-batch training was employed with a batch size of 16. The model was trained for 100 epochs, with dropout applied during training to improve generalization.

To monitor performance during training, the model was evaluated on the validation set after each epoch using macro F1-score. This metric was selected because it gives equal importance to all classes and is therefore more suitable for multi-class classification problems where class performance must be assessed uniformly. The model checkpoint achieving the best validation macro F1-score was saved and used for final evaluation.

In summary, the training strategy combines weighted cross-entropy loss, Adam optimization, dropout-based regularization, and validation-based checkpoint selection to improve the robustness and generalization of the proposed graph-based component classifier.

A built-in scheduler(ReduceLROnPlateu), reduces learning rate if validation scores stagnate. Training halts early when patience is exceeded. Afterwards, the script loads the best checkpoint and performs a full test-set evaluation of the Weighted F1-scores for node and graph level, and provides full classification reports and confusion matrices.

On the higher-level, GraphSAGE works via Sampling and Aggregation. Unlike earlier GNNs such as GraphConv, GraphSAGE does need to load the entire graph, and can be uniformly trained using a fixed sample size of neighbors. This is then aggregated into a single neighborhood vector in conjunction with the node’s own feature vector and is then passed through ReLU to create a new node embedding. This approach makes GraphSAGE computationally efficient, but can handle new and unseen nodes without retraining via neighborhood aggregation as seen in Figure 3

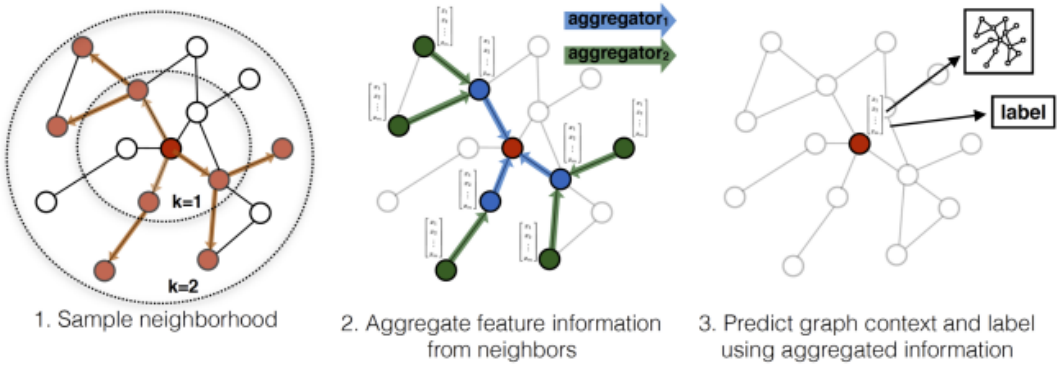

Figure 3. GraphSAGE high-level diagram

Rules-Based System Code Transpilation Architecture

The NeoDev system utilizes a rules-based transpiler implemented as a high-performance microservice using the FastAPI framework. This component serves as the final stage of the pipeline, converting the structural JSON representation of a user's design into framework-specific source code.

Service Design and Safety Constraints

The transpiler is designed to handle complex DOM structures while maintaining system stability. To prevent

resource exhaustion during the recursive traversal of deep trees, the service enforces several operational limits:

- Max Node Count: Restricted to 5,000 nodes per request.
- Recursion Depth: Limited to 100 levels to prevent stack overflow errors during tree traversal.
- Input Size: A maximum payload limit of 500,000 bytes is established for incoming HTML/JSON data.

HTML-to-AST Intermediate Representation

Before framework conversion occurs, the system parses the input into an intermediate Abstract Syntax Tree (AST) using the BeautifulSoup library. This stage normalizes the raw user input into a standardized node structure. Each node in this AST contains:

- Type: The normalized lowercase HTML tag name.
- Attributes: A dictionary of properties, where key attributes like id are prioritized for structural clarity.
- Styles: An OrderedDict of CSS properties to ensure styling consistency.

- Children: A recursive list of nested nodes, preserving the hierarchical relationship identified by the GraphSAGE classifier.

Style Management and Deduplication

A core feature of the transpiler is the StyleManager, which implements an automated CSS deduplication algorithm.

1. Registration: As the AST is traversed, the manager captures inline style declarations.
2. Hashing: It serializes style sets and generates a unique MD5 hash for each unique combination of properties.
3. Class Generation: Identical styles are assigned a shared generated class name (e.g., .c1, .c2). This process ensures that the generated code is optimized, replacing repetitive inline styles with clean, reusable CSS classes, thereby reducing the final file size.

Framework-Specific Backend Engines

The system employs two distinct rendering backends to generate production-ready code for different frontend ecosystems:

- React Backend: This engine performs "syntax-shimming" to ensure JSX compatibility. It automatically maps standard HTML attributes to their React equivalents (e.g., converting class to className and for to htmlFor). Additionally, it transforms CSS string declarations into camelCase JavaScript objects, which are then injected into the style prop of the component. The final output is wrapped in a standard functional component export.
- Svelte Backend: This engine generates standard-compliant HTML markup. Unlike the React backend, it utilizes a scoped architecture by appending a <style> block at the end of the file. This block contains the compiled CSS rules generated by the StyleManager, resulting in a clean separation of concerns within a single-file component (.svelte).

Procedural Logic for Void Tags

To ensure the generated markup is syntactically valid, the transpiler maintains a definition of Void Tags (e.g., <img>,
, <hr>). During the recursive reconstruction, the system identifies these tags and automatically applies self-closing syntax, preventing the creation of broken or illegal HTML structures in the final exported project as shown in Figure 4

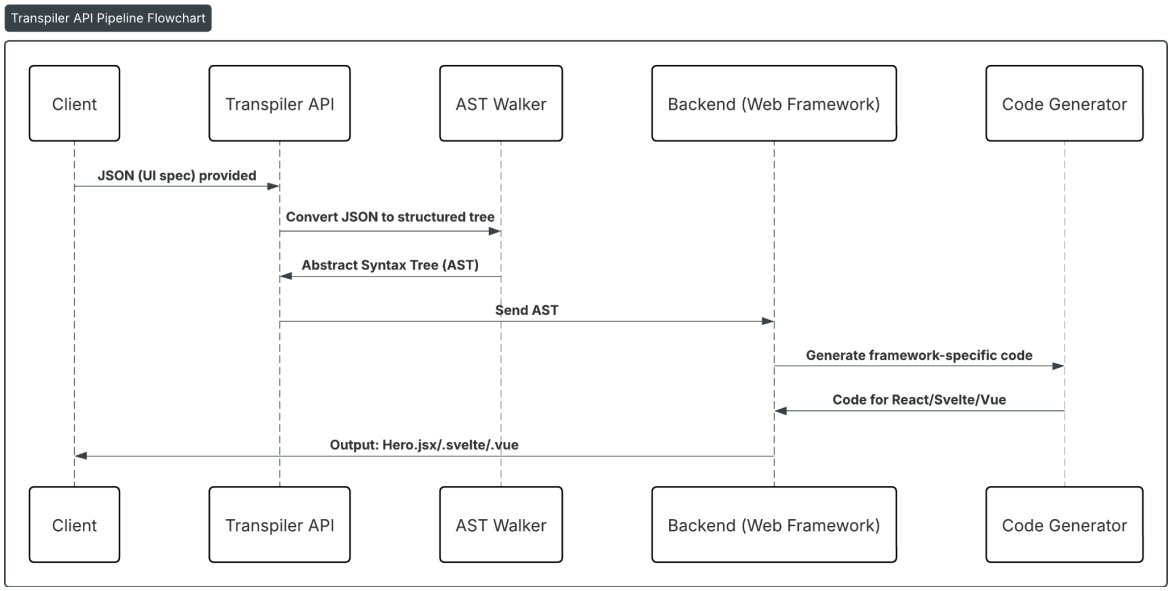

Figure 4. Transpiler API Pipeline

Components and Design

System Architecture

The NeoDev website is built using React with Tailwind CSS and TypeScript, and is hosted on Vercel. User data is stored locally on their device through locaStorage. It uses a powerful system to track and update our app's data in real time, so our web pages automatically update when we change things. This is important for keeping track of what our user is doing or seeing in NeoDev, which is important for storing the current state of the user's project. Tailwind CSS is a utility-first CSS framework that lets us build designs fast without writing custom CSS at all. TypeScript is a superset of JavaScript that adds strict typing and rules about what kind of data JavaScript can use. Vercel is a free provider for hosting websites that lets us put our web development learning tool online with features that help it run smoothly and securely.

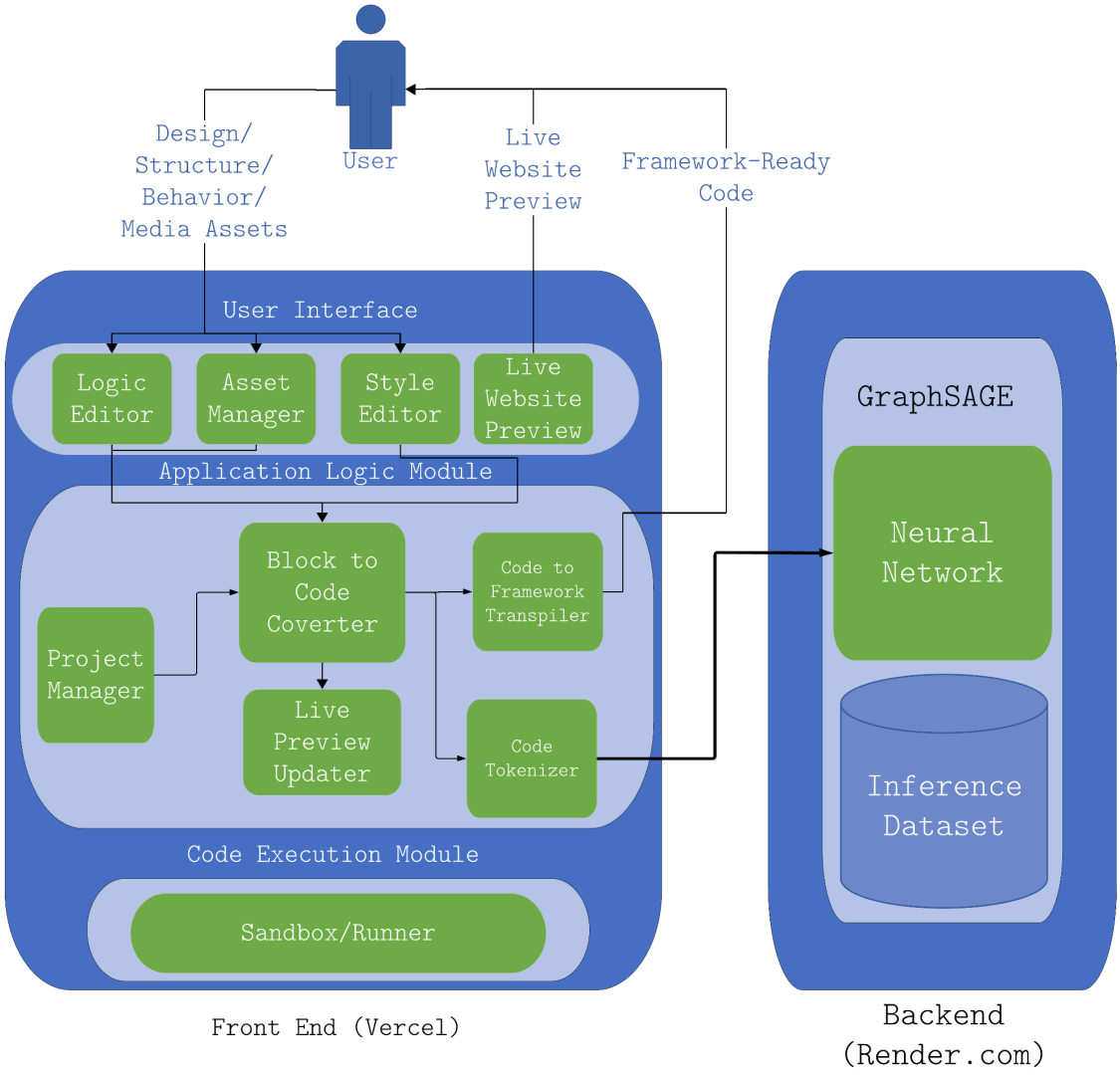

Figure 5. System Architecture of the Application

Figure 5 shows the structure of our system using the Front-End-First System Architecture. Our main frontend app, built with React, is hosted on Vercel. The backend server that hosts the GraphSAGE model is written in Python and

FastAPI. The prediction route for the GraphSAGE Classifier is specified as /predict, and the deployment of the backend server is hosted on the same server as the frontend.

The main web application is divided into three modules. The first module contains all the UI elements and is responsible for interacting with the user. Afterward, it takes user input in the form of blocks using the playground and the properties panel. Additionally, on this layer is a Live Website preview that allows the user to track their progress without needing to view the code.

The next layer, called the application layer, contains most of the important functions of the frontend application. It is in this layer where the blocks are converted to code before being tokenized and passed to the GraphSAGE Neural Network Classifier. GraphSAGE then returns a classification/identification of what the code snippet is, as well as insights via the “Did You Know?” component on the frontend. Additionally, the blocks are passed into the Code-to-Framework transpiler, which converts the blocks into an intermediary form, and subsequently into a working front-end framework via a set of predefined rules.

Additionally, the Live Preview Updater is also located here and ensures that the Live Preview is constantly updated.

The last module is a Sandbox/VM layer, which serves as an isolated execution environment—utilizing browser-based virtualization—to build and render the Live Website Preview in real-time.

The purpose of GraphSAGE in the system is to act primarily as a classifier that identifies the specific component in the website, and then provides insights and warnings based on that component. The Code Blocks are converted into proper code before it is passed into the tokenizer. For classification, the code is tokenized and converted into a Hierarchical Tree Structure in the JSON file format before it is passed to GraphSAGE. Data is then pre-processed and one-hot encoded before being passed to the Neural Network Classifier. The Classifier then returns warnings and insights based on the identified component to the user. Upon export to a front-end framework, GraphSAGE also adds optimizations and passes them on to the code transpiler.

Application Storage and Design

For the Database, the researchers implemented a Local File Management approach instead of relying on external database services. This method stores user accounts and project data directly within the system's local environment, enabling users to save their progress without the need for a cloud-based backend.

At the highest level, we have the user data entity, which contains the user's ID, Username, and Project, which is a list of Project lists associated with the user. Next, we have the project entity, of which a user may have multiple projects. This contains a project ID, as well as the User ID to tie it to its creator, a project title, and a timestamp of when the project was last opened.

Each project also includes many Project_Component entities, which serve as a one-to-one bridge between a certain Component and the Project that a specific iteration of a component is tied to. The Component entity contains identifiers such as Component ID, Component Type, Component Name, and a label. These correspond to the various tags found in the structure and style editor.

Each component has an attribute entity, which describes the basic structure of the component and corresponds to its intended equivalent in HTML (E.G.: Header, Body, Paragraph, Image); A style entity, which describes how the visual appearance of the component (E.G.: Font Size, Style, and Color) and corresponds to its intended CSS Styling; and a Children Entity, which ties any components that are nested within a component to said component and helps define the structure of a website (E.G.: A paragraph component inside a header component)

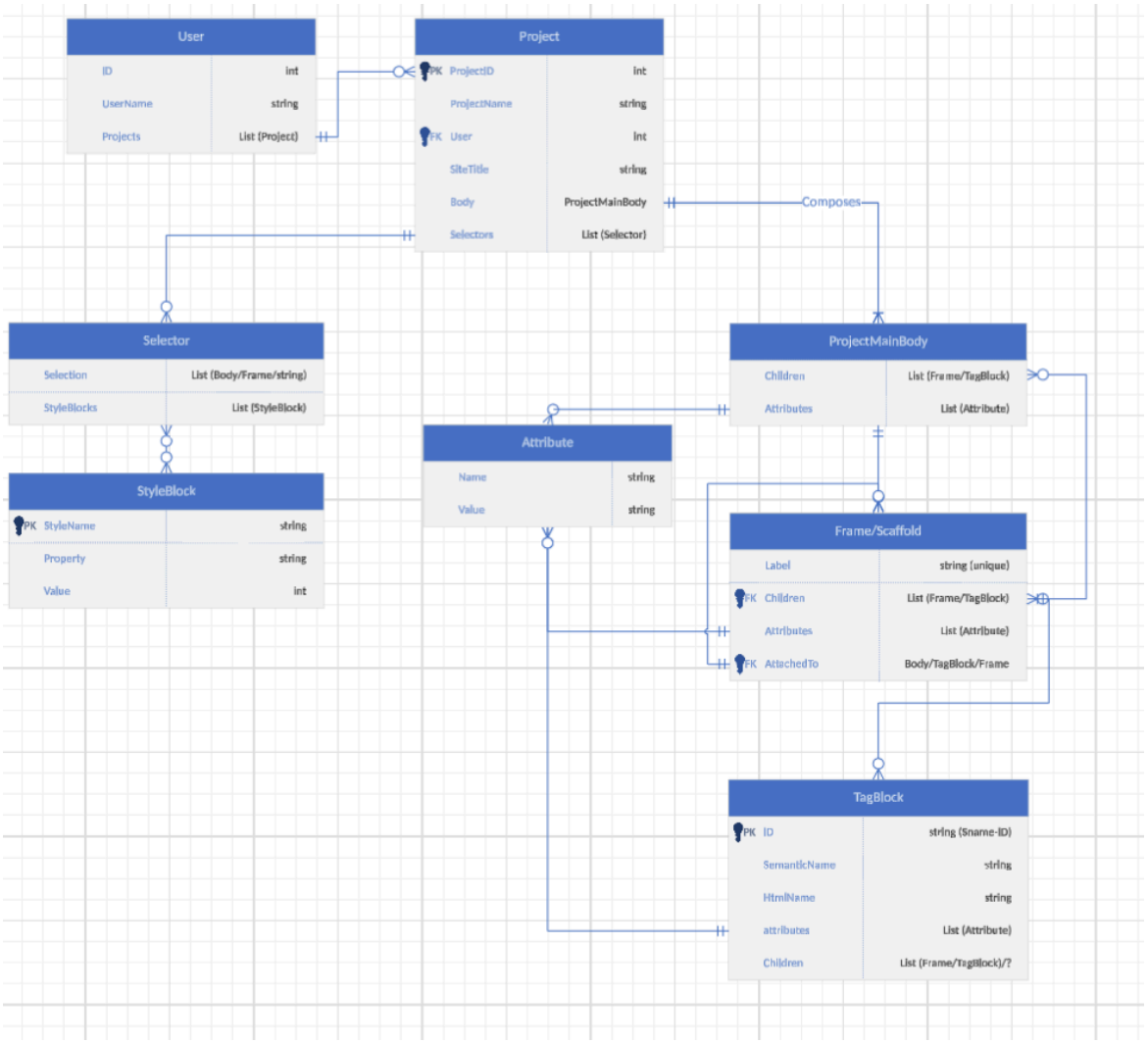

Figure 6. ERD relational diagram for the JSON file

Model Architecture

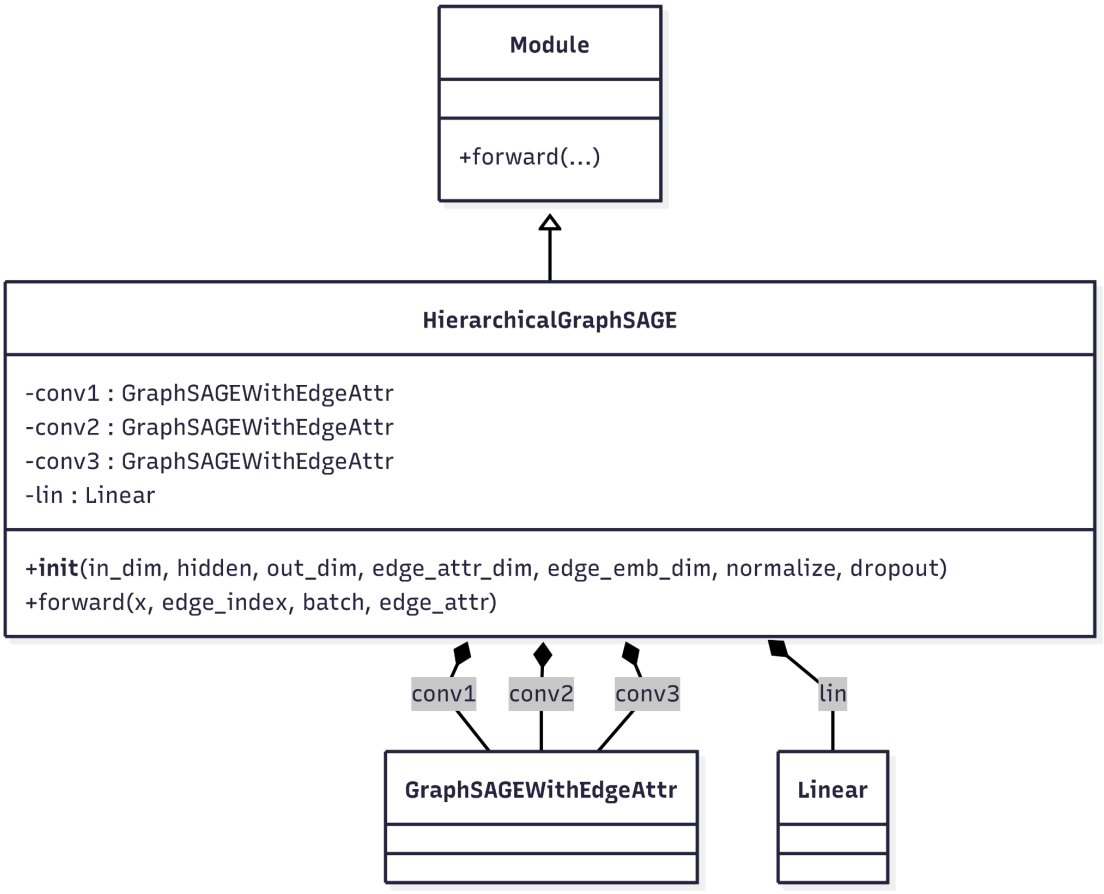

Figure 7. GraphSAGE UML-Class Diagram

Figure 8 describes the complete GraphSAGE model, which is composed of three sequential GraphSAGEWithEdgeAttr convolutional layers (conv1, conv2, and conv3) followed by a global mean pooling layer and a final linear classifier. The model takes as input node features, graph connectivity, edge

attributes, and batch assignments, then produces one output vector for each graph in the batch.

This model has four inputs: the node feature matrix X, the edge index matrix E, the edge attribute matrix, and the batch vector. Node features encode per-node attributes, while edge attributes provide relational information during message passing. Each layer performs edge-aware message passing by combining neighboring node representations with learned embeddings of the corresponding edge attributes.

The aggregated neighborhood representation is then added to a transformed self-representation of the target node. Following that, the three layers progressively refine node embeddings by expanding the receptive field. As there are three layers, each final node embedding contains information propagated from up to three hops away in the graph. After node embedding, a global mean pooling operation aggregates all node representations belonging to the same graph into a single fixed-length graph embedding. This readout step enables the model to perform graph-level classification regardless of graph size. The pooled graph

representation is passed through a final linear layer to produce the output logits for classification.

To improve generalization, dropout is applied after the first two convolutional layers and again after graph-level pooling. Optional normalization inside each layer stabilizes intermediate feature distributions during training.

GraphSAGEwithEdgeAttr Class

GraphSAGEWithEdgeAttr is a custom graph convolution layer based on the GraphSAGE message-passing framework. Unlike standard GraphSAGE implementations that aggregate only neighboring node features, the proposed layer incorporates edge attributes into the message construction process. This allows the model to learn node representations that depend not only on neighboring nodes but also on the relationships connecting them. Figure 9 illustrates the class as a UML Diagram.

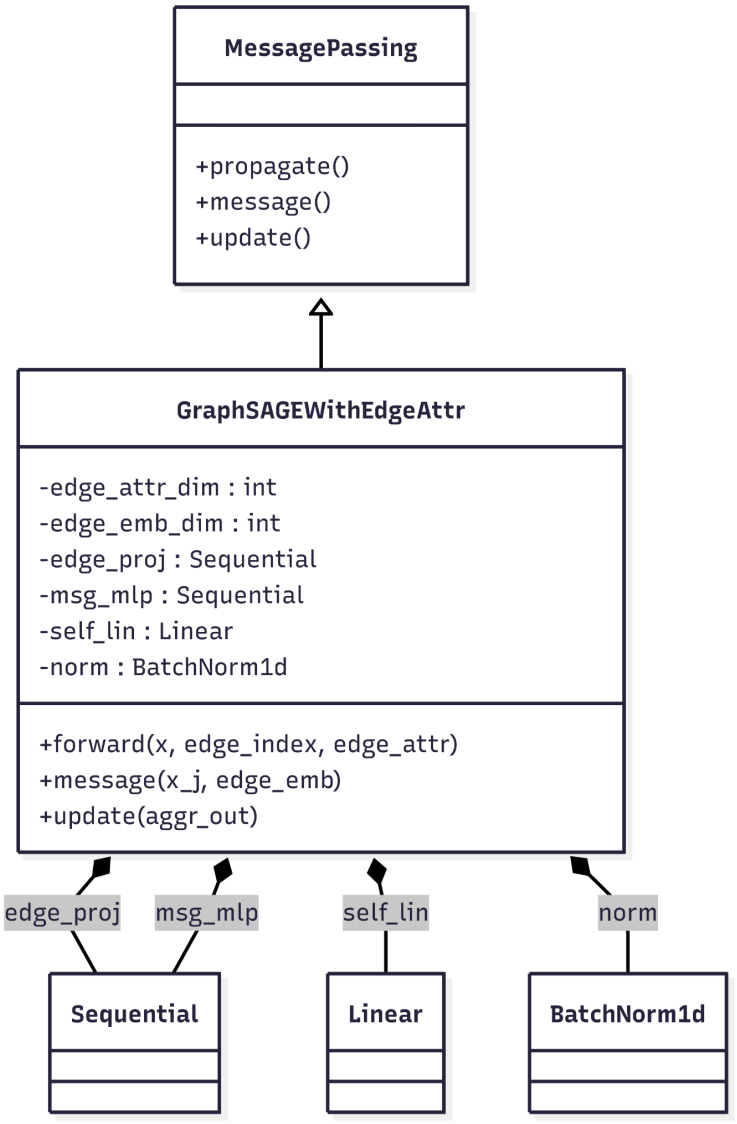

Figure 8. GraphSAGEwithEdgeAttr UML Diagram

The layer takes node feature matrix , graph connectivity E, and the edge attribute matrix as its inputs. It outputs an updated node embedding matrix where each node

representation reflects both its own features and the aggregated contributions of its neighbors through edge-aware message passing.

To make edge information more expressive, the raw edge attributes are first transformed into learned edge embeddings using a small multilayer perceptron. This projection step enables the model to map raw edge descriptors into a latent space more suitable for message construction. If edge attributes are unavailable, the layer uses zero-valued edge embeddings, allowing the same message-passing mechanism to operate even in the absence of explicit edge features.

For each edge, the message is constructed by concatenating the neighboring node representation with the corresponding edge embedding. This concatenated vector is then passed through a message multilayer perceptron to produce an edge-aware message. The messages from all neighboring nodes are aggregated using mean aggregation. This produces a single neighborhood representation for each node while keeping the embedding scale relatively stable across varying neighborhood sizes.

In addition to the aggregated neighbor information, the layer learns a linear transformation of the node's own feature vector. This self-representation preserves node-specific information and allows the updated embedding to balance local neighborhood context with the node's original attributes. The final node representation is obtained by combining the transformed self-feature term with the aggregated neighborhood message. Optional batch normalization is then applied to stabilize training. An optional batch normalization layer is applied after node update to stabilize feature distributions during training. and improve optimization stability during training.

User Workflow

As shown in the flowcharts in figures 10 and 11, the user workflow starts at the landing page, before it proceeds to the playground. Everything happens in real-time as while the user builds the website using the block editor, HTML and CSS code is also displayed in real time in a smaller separate window. At the same time, this code is passed to the GraphSAGE trained inference API which analyzes the structure of the code and returns a result. This result is then passed to a large language model, which provides insights and tips based on the given result. Once the user is satisfied with the result, they can then save the code and download it as a fully-ready file of their chosen framework via rules-based transpilation that adds fixes, optimizes code, and ensures that it is deployment-ready.

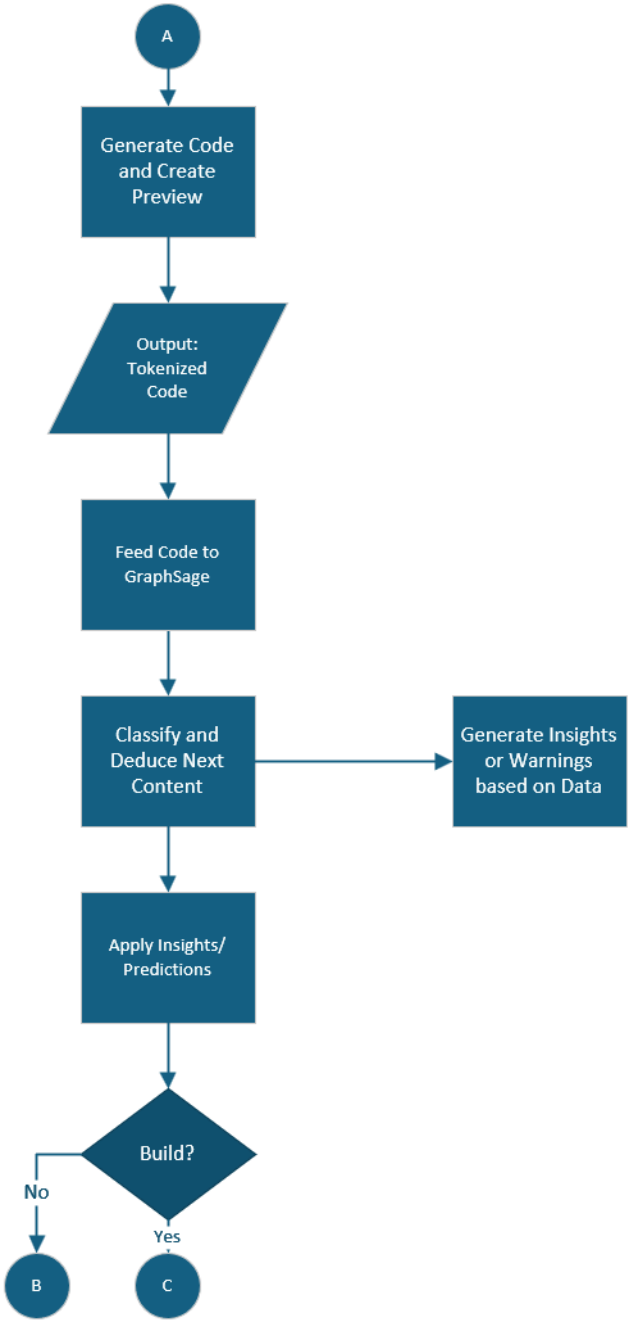

Figure 9. User Workflow Flowchart (Part One) - Illustrates the program workflow from start to finish

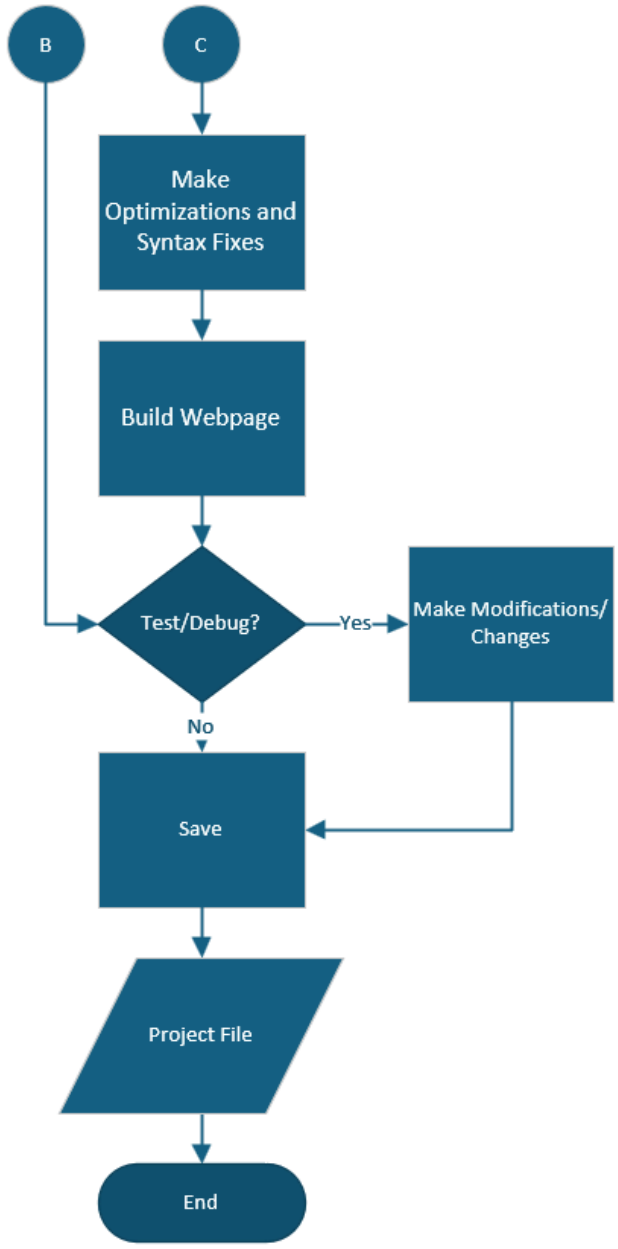

Figure 10. User Workflow Flowchart (Part Two) - Illustrates the program workflow from start to finish

Methodology

System Development Life Cycle

The development of the system followed an Iterative Process Model, characterized by successive cycles of design, implementation, and evaluation. This approach allowed for the continuous refinement of the system's architecture, particularly in the integration of the GraphSAGE model and the optimization of the user interface.

Iterative Development Phases

The project was executed across three primary iterations, each targeting specific functional milestones:

Iteration I: Requirements and Prototyping – This initial cycle focused on the elicitation of system requirements and the construction of the primary architectural prototype.

Iteration II: Integration and Communication – The second phase addressed the integration of backend services and established the communication protocols between the frontend framework and the GraphSAGE model.

Iteration III: Refinement and Finalization – The final cycle was dedicated to rigorous bug mitigation, performance tuning, and the finalization of the system's feature set.

Project Timeline and Implementation

Planning and Analysis (January 2025)

The planning phase involved the definition of core workflows and the establishment of technical governance, including Git contribution protocols and repository management strategies. During the subsequent implementation analysis, React was selected as the primary development framework due to its efficient component-based paradigm and extensive ecosystem support. Concurrently, initial construction and feasibility testing of the GraphSAGE model were performed to validate the project's technical core.

Design and Initial Implementation (February 2025)

The design phase finalized the user interface (UI) and structural layouts within Figma. Implementation commenced in late February, focusing on translating the finalized designs into functional components. This three-week development

sprint resulted in a minimum viable prototype featuring active GraphSAGE integration.

System Refinement (April – May 2025)

The second iteration of the project focused on enhancing the user experience (UX). While the core feature set remained stable, the UI underwent significant restructuring to improve usability and streamline user interactions. This period ensured that the technical backend was supported by an intuitive and accessible frontend interface.

System Finalization (August 2025 - January 2026)

System development continued after summer break. During this time the front-end was revised while the model continued further training. At this time, the processing, training, and inference process was revised, with the model shifting towards a hierarchical approach that would allow it to take better advantage of DOM structures. The front-end and website was also revised with major additions to the layout and the addition of a landing page. A Large Language Model was also integrated into the application around this

time to produce user-friendly insights from the raw GraphSAGE results. The system was finalized in december, and was ready for open beta testing by January 2026

CHAPTER 4 RESULTS AND DISCUSSION

Implementation

Development Tools and Environment

The architectural planning and interface design were conducted using Figma and Excalidraw, facilitating structured layout prototyping and collaborative diagramming. Project milestones, synchronous communication, and progress reporting were managed via Google Meet.

Technical Stack and Deployment

The project's primary technical stack comprises React and Tailwind CSS for the frontend, utilizing the latter for its granular customization and responsive design capabilities. Python served as the core language for backend logic, specifically for the implementation of the GraphSAGE algorithm and the construction of custom transpilers designed to convert JSON structures into framework-ready frontend code.

To ensure global accessibility and high availability of the backend services, the API endpoint was deployed using Render, a cloud-based web service platform. Version control and source code management were maintained through a private GitHub repository using Git.

Implementation Environment

Development was performed within Visual Studio Code (VS Code), integrated with a specialized suite of extensions—including ES7+ React/Redux/GraphQL Snippets, Tailwind Fold, Tailwind IntelliSense, and Prettier—to optimize workflow efficiency. These tools facilitated the seamless integration of React, the project's primary framework, while the bun runtime was utilized to power the JavaScript execution environment and streamline the build process.

Client-Side Hardware Requirements

The system architecture adopts a thin-client approach, where computationally intensive operations are offloaded to the server. Consequently, the client-side hardware requirements are minimal. The web application is compatible

with any standard computing system manufactured after 2010, provided it utilizes a modern, updated web browser.

To ensure an optimal user experience, the recommended hardware specifications are detailed in table 1 located below:

Component	Minimum	Recommended
CPU	Any	2 Cores
GPU	Integrated Graphics	Dedicated Graphics Card
RAM	2 Gigabytes	8 Gigabytes
Storage	Any	Any SSD (128 GB and up)
Internet Connectivity	Exists	10-20 Mbps
Peripherals	Mouse and Keyboard	Mouse, Keyboard, Speaker

Table 1. Minimum Recommended Client Side Specs

Server-Side Infrastructure and Requirements

The server-side infrastructure is designed to manage high-concurrency tasks, including user authentication, real-time website generation, and the transpilation of source code into frontend-ready frameworks. To maintain

performance during these resource-intensive operations, a distributed hosting strategy is employed.

Cloud Deployment

The production environment is currently decoupled across two primary platforms:

- Frontend Hosting: The user interface and client-side logic are deployed via Vercel to ensure global edge delivery and rapid deployment cycles.
- Computational Backend: The GraphSAGE implementation and associated logic are hosted on Render, providing a dedicated environment for executing complex graph-based operations and API requests.

Local Hosting and Hardware Specifications

For local deployment or self-hosted environments, the hardware requirements scale according to the concurrent user load. While the system can function on a baseline configuration, the resource-intensive nature of the compilation transpilers and graph processing necessitates significant memory and processing power as noted in table 2, located on the next page.

Component	Minimum	Recommended
CPU	4 cores	8 cores
RAM	32 gigabytes	64 Gigabytes
Storage	SSD (512 GB and up)	NVME SSD (1 TB and up)
Internet Connectivity	100-200 Mbps	2.5 gigabit LAN

Table 2. Hosting Requirements

Software Specifications

The project was developed using a modern software ecosystem designed for high performance and scalability. The core development environment and deployment platforms are categorized as follows:

Development Frameworks and Languages

The application was built using React 19.2 for component-based architecture and Tailwind CSS 4.1 for utility-first styling. The backend logic and data processing modules were implemented using Python 3.7. All source code was authored within Visual Studio Code (VS Code), utilizing its latest stable release to leverage contemporary debugging and extension capabilities.

Deployment and Cloud Infrastructure

The system architecture utilizes a distributed deployment strategy to optimize performance across different application layers:

- Frontend Deployment: The client-side application is hosted on Vercel, a cloud platform optimized for frontend frameworks. Vercel's automated deployment pipelines and global edge caching, ensuring low-latency access for users.
- Backend Hosting: Due to the specific computational requirements of the GraphSAGE classifier, the backend environment is hosted on Render. Render provides the necessary infrastructure for persistent web servers and manages the execution environment for the Python-based API logic.

Client-Side Compatibility

Access to the web application is platform-agnostic, requiring only a modern desktop or laptop-based web browser. The system is compatible with any browser that supports HTML5 and CSS3 standards (e.g., Chrome, Firefox, Edge, or Safari). This ensures that the application remains accessible across various operating systems without the need

for locally installed dependencies. Table 3 here showcases this along with the necessary tech stack to operate.

Stack	Version	Purpose
React	19.2	Primary Frontend Framework (Structure and Styling for UI)
Tailwind	4.1	Additional CSS Modules and Functionality
Python	3.7	GraphSAGE, Framework-to-Code Transpilation
Vercel	N/A	Web Application Hosting
Any Web-Browser	Chrome: 130 Firefox: 130 Edge: 130 Safari: 17 Opera: 107	Application Access

Table 3. Current Tech Stack

Data Collection

The data collection phase involved the systematic extraction and manual labeling of Document Object Model (DOM) elements. This was facilitated by a custom-developed web-scraping and annotation utility, built with a Python Flask backend and a JavaScript frontend. The backend logic, managed by app.py, defines the categorical labels for DOM

identification and orchestrates the data serialization pipeline as seen in figure 15.

```
IMPORT JSON library

   CREATE web application

   DEFINE LABELS as:
       DropdownMenu, SearchBar, RangeSlider, Card, ImageGallery,
       Link, NavigationBar, Table, IconButton, Header, Footer, Sidebar

   DEFINE route "/" for homepage
       WHEN homepage is requested:
           RENDER "index.html"
           PASS LABELS to the template

   DEFINE route "/submit" accepting POST requests
       WHEN submit request is received:
           READ JSON data from request
           OPEN file "labeled_data2.json" in write mode
           SAVE JSON data into file with indentation
           RETURN success response with message "Data saved."

   IF program is run directly:
       START web application in debug mode
```

Figure 11 Dataset Maker App.py as pseudocode

The interface is divided into two primary functional areas. The input area allows for the insertion of raw HTML snippets, while the control panel manages the session state through functions such as "Start Labeling," "Undo," and "Submit." This design allows the researcher to observe a live visual representation of the component before assigning a label (see Figure 15).

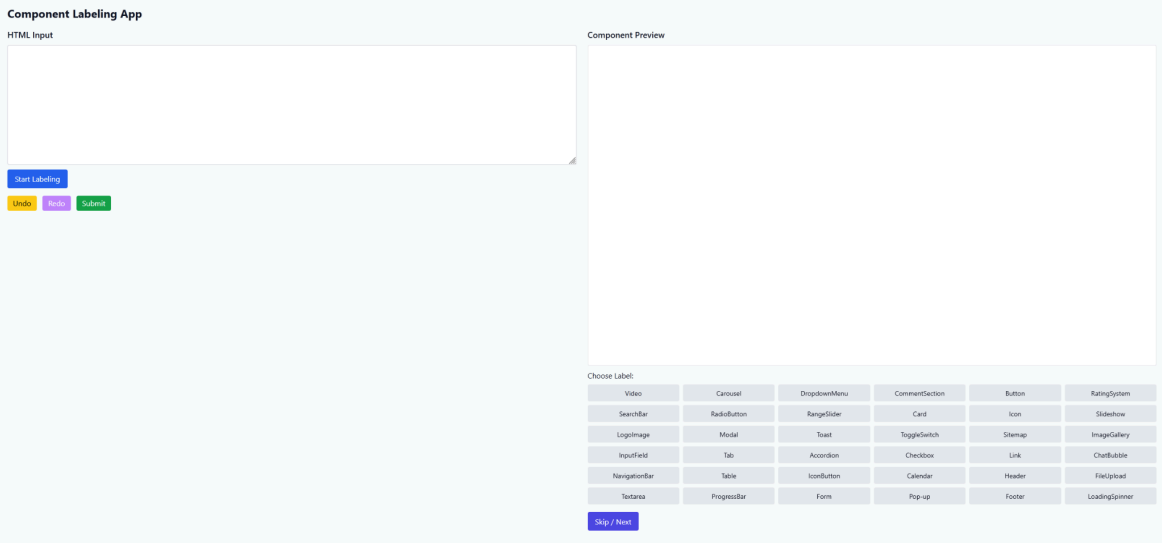

Figure 12. WebScraper UI

Data Collection Procedure

The labeling workflow followed a structured protocol:

1. Source Selection: Targeted websites were selected based on lightweight architectures to ensure high-fidelity extraction.

2. Extraction: Raw HTML was copied from the browser's developer tools and transferred to the annotation utility.
3. Manual Annotation: Components were assigned specific labels based on structural design. In cases of

ambiguity, the "skip" function was utilized to maintain dataset integrity.

4. Serialization: Upon completion, the "Submit" function serialized the data into a structured labeled_data.json file.

Data Augmentation

To address the limitations of manual data collection and to improve the model's generalization across various design styles, a specialized Synthetic Data Augmenter was developed. This tool programmatically generates diverse UI components by simulating different CSS configurations and structural variations common in modern web development.

The generation logic, as shown in Figure 16 employs a predefined schema of common UI elements—including Navigation Bars, Data Tables, and Image Galleries. Each component is dynamically assembled using a recursive node-creation method. This method ensures that every generated element maintains a strict mapping between its HTML tags, attribute classes, and metadata identifiers.

```
Inputs
Label (L) — the Ul component type to generate
Set of Styles (5) — includes colors, shadows, and rounding

Output
Synthetic DOM Node (N)

Procedure
Initialize an empty list of children.

Conditional Generation Logic

If L is "NavigationBar”:
Generate a random number of anchor nodes.
Assign randomized text colors to each anchor.
Create a parent header element.
Apply a random background color and a random shadow from S to the header.
Insert a title node and the generated anchor nodes into the header.
Return the header containing all children.

Else if L is "Card":
With probability P, append an image node to the children list.
Append a div containing a text node and an optional button.
Apply a random shadow and a random corner radius from S.
Return the div container with its children.

Else if L is “Table”:
Generate R rows, where R is a random integer.
For each row, generate C columns.
Apply border utility classes to each column.
Return a table node containing the generated rows.

Else (Fallback case):
Map L to its corresponding standard HTML tag.
Apply a random color from S.
Return the basic node.

End Result
A label-driven synthetic DOM node with randomized styling suitable for UI generation or dataset creation.
```

Figure 13. Logic for Synthetic Component Generation

To introduce realistic variance into the dataset, the script applies stochastic (randomized) design attributes to each component:

- Structural Variance: Elements like tables and galleries are generated with a randomized number of rows,

columns, or child nodes to simulate varying content density.

- Stylistic Permutations: Visual attributes, such as background colors, shadow depths, and border-rounding (radius), are sampled from a distribution of standard Tailwind CSS utility classes.
- Compositional Logic: Complex components like Cards are generated with optional sub-elements (e.g., images or buttons) based on probabilistic thresholds, mimicking the diversity found in real-world web interfaces.

Dataset Expansion The utility generates a balanced batch of synthetic samples for each supported label category. By automating the creation of hundreds of unique instances per component, the augmentation process provides the diverse structural patterns necessary for the GraphSAGE model to learn generalizable features rather than memorizing specific HTML snippets. A sample of which can be seen on Figure 17

```
{
  "label": "DropdownMenu",
  "contents": [
    {
      "type": "div",
      "attributes": { "class": "relative" },
      "children": [
        {
          "type": "button",
          "attributes": { "class": "px-4 py-2 bg-white border shadow-sm" }
        },
        {
          "type": "div",
          "attributes": {
            "class": "absolute top-full mt-2 w-48 bg-white border shadow-xl"
          },
          "children": [
            { "type": "div", "attributes": { "class": "p-2 hover:bg-gray-100" } },
            { "type": "div", "attributes": { "class": "p-2 hover:bg-gray-100" } },
            { "type": "div", "attributes": { "class": "p-2 hover:bg-gray-100" } }
          ]
        }
      ]
    }
  ]
}
```

Figure 14. Synthetically Generated Component Output Sample

Following the initial generation phase, the synthetic dataset reached a total of 6,000 individual samples, distributed equally with 500 instances per label. However, a formal validation pass using a uniqueness checker revealed significant structural redundancy. Despite the volume of data, several label categories exhibited a high degree of uniformity, with some classes containing only a single

unique structural variation across all samples as shown in Table 4.

Label	Total	Unique Variations
Card	500	96
DropdownMenu	500	1
Footer	500	5
Header	500	5
IconButton	500	5
ImageGallery	500	12
Link	500	5
NavigationBar	500	389
RangeSlider	500	5
SearchBar	500	5
Sidebar	500	1
Table	500	3
Total Samples	**6000**	—

Table 4. Dataset Uniqueness Report

Heuristic Mutation and Structural Diversification

To address this lack of variance and prevent model overfitting, a second stage of data augmentation was implemented. This phase utilized stochastic mutation to enhance the diversity of the dataset through the following methods:

1. **Attribute Shuffling:** CSS utility classes and HTML attributes were dynamically reordered and swapped to create unique stylistic permutations for identical component types.
2. **Hierarchical Transformation:** The relationships between parent and sibling nodes were intelligently modified. By shifting tags and redistributing nested elements, the utility generated new architectural patterns while maintaining the functional integrity of the UI component.

This iterative refinement process ensured that the final dataset provided the structural entropy required for the GraphSAGE model to effectively learn complex spatial

relationships within the DOM, the logic of which can be found on Figure 19 on the next page.

```
function mutate_classes(attrs):
    if "class" not in attrs:
        return attrs

    classes = split(attrs["class"])

    if classes is empty:
        return attrs

    shuffle(classes)

    if length(classes) > 1:
        classes = [c for c in classes if random() > 0.10]

    attrs["class"] = join(classes, " ")

    return attrs
```

Figure 15. Mutate Classes Function for attributes

After mutating the classes and sibling relationships (see Figure 16), the number of unique samples for each label increased with the card label now having 498 unique variations compared to 96 initially. DropdownMenu went from one unique sample replicated 500 times to 500 unique samples for the label. This dataset was saved as

tailwind_augmented.json and was used as the final dataset for training inference (see Table 5).

Label	**Total**	**Unique Variations**
Card	500	498
DropdownMenu	500	500
Footer	500	24
Header	500	23
IconButton	500	23
ImageGallery	500	500
Link	500	24
NavigationBar	500	500
RangeSlider	500	23
SearchBar	500	497
Sidebar	500	500
Table	500	500

Table 5. Uniqueness Report after mutator pass-through

Additionally, a secondary, stratified dataset was also created for the purposes of testing potential data

saturation issues. This dataset was greatly reduced in terms of sample size and only had 480 graphs out of 6000, with 40 samples for 12 labels. This dataset was then saved as tailwind_small.json, the logic of which can be seen on the next page in Figure 16.

```
Algorithm: Balanced Random Subset Creation

Input:
    INPUT_FILE = source JSON dataset
    OUTPUT_FILE = destination JSON file
    SAMPLES_PER_CLASS = desired number of samples per label

Output:
    A shuffled, class-balanced subset written to OUTPUT_FILE

1. Read the JSON dataset from INPUT_FILE.
2. Group all samples according to their label.
3. Initialize an empty subset list.
4. For each label group:
   4.1 If the number of samples is less than SAMPLES_PER_CLASS, add all samples to the subset.
   4.2 Otherwise, randomly select SAMPLES_PER_CLASS samples and add them to the subset.
5. Shuffle the final subset.
6. Write the subset to OUTPUT_FILE in JSON format.
7. End.
```

Figure 16. Dataset reduction tool

Pre-Processing

Following the data augmentation phase, the dataset underwent a multi-stage pre-processing pipeline to transform raw JSON DOM structures into graph-based representations suitable for deep learning.

Feature Extraction and Vectorization

The pipeline initiates with a Depth-First Search (DFS) algorithm to recursively traverse the DOM hierarchy. As the algorithm traverses the structure, it flattens the nested

hierarchy into a discrete list of nodes and edges while extracting three primary feature categories:

1. Semantic Features: These include one-hot encoding of HTML tags and the vectorization of CSS classes. The latter utilizes Term Frequency-Inverse Document Frequency (TF-IDF) to capture stylistic and functional terminologies, transforming text-based styling into a numerical feature space.
2. Structural Features: The algorithm computes intrinsic properties such as node depth, leaf status, and child count. A secondary pass executes subtree calculations to determine descendant density and the concentration of interactive or media elements.
3. Heuristic Flags: Binary flags are assigned based on specific attributes (e.g., accessibility ARIA labels), providing the model with explicit signals regarding an element's configuration.

Figure 22 on the next page showcases the following as the helper functions, which get the root node, collect tags from the dataset, as well as the flags of whether it’s interactive, it’s a media component or element, and any other attribute flags

```
Algorithm: DOM Root Extraction and Attribute Analysis

1. Define a function to extract the root node from an item:
   1.1 If the item is a list, return its first element if available.
   1.2 Otherwise, check whether the item contains a "dom" or "contents" field.
   1.3 If neither exists but the item itself is already a node, return the item.
   1.4 If the extracted root is a list, return its first element.
   1.5 Return the resulting root node.

2. Define a function to collect tags from the dataset:
   2.1 Initialize an empty set of tags.
   2.2 Traverse each root node using depth-first search.
   2.3 For each visited node, extract its tag, type, or name value.
   2.4 Convert the value to lowercase and store it in the tag set.
   2.5 Recursively process all child nodes.
   2.6 Return the sorted list of unique tags.

3. Define a function to detect interactive elements:
   3.1 Return true if the attributes contain "href" or "onclick".
   3.2 Also return true if the tag is one of: button, input, select, textarea, or a.

4. Define a function to detect media elements:
   4.1 Return true if the attributes contain "src".
   4.2 Also return true if the tag is one of: img, video, audio, or svg.

5. Define a function to generate attribute flags:
   5.1 Check for the presence of class, id, style, href, src, and role.
   5.2 Check whether any attribute key begins with "aria-".
   5.3 Return these checks as binary indicator values.
```

Figure 17. Pre-Processing Code Snippet: Helper Functions

Graph Topology Construction

Once features are vectorized, a PyTorch Geometric (PyG) data object is constructed. The system defines the DOM topology using a tri-directional edge schema to ensure comprehensive message passing within the GraphSAGE

architecture. As demonstrated in the Node and Edge creator logic (see Figure 23), the schema consists of:

- Parent-Child (Type 0): Directed edges flowing from container to content.
- Child-Parent (Type 1): Reverse edges allowing information flow back up the hierarchy.
- Sibling-Sibling (Type 2): Bi-directional edges connecting adjacent nodes at the same depth.

```
Algorithm: DOM Traversal and Subtree Statistics Computation

Input: Root node of a DOM tree
Output: A list of node records, a list of graph edges, and subtree statistics

1. Initialize empty lists for nodes and edges.
2. Perform a depth-first traversal starting from the root node.
3. For each visited node:
   3.1 Extract its tag, attributes, children, parent index, and depth.
   3.2 Create a node record containing:
       - tag
       - attributes
       - depth
       - parent reference
       - child list
       - leaf indicator
       - interactivity indicator
       - media indicator
   3.3 Add the node record to the node list.
   3.4 If the node has a parent:
       - Add a parent-to-child edge
       - Add a child-to-parent edge
       - Register the node as a child of its parent
   3.5 Recursively visit all child nodes.

4. After traversal, add sibling edges:
   4.1 For each node, inspect its child list.
   4.2 Connect each child to its next sibling in both directions.

5. Return the complete node list and edge list.

6. To compute subtree statistics:
   6.1 If no nodes exist, return an empty result.
   6.2 Starting from the root node, recursively count:
       - total descendants
       - interactive descendants
       - media descendants
   6.3 Return these counts as subtree statistics.
```

Figure 18. Pre-Processing Code Snippets – Node and Edge creators

This multi-type edge configuration allows the model to capture layout context both vertically (hierarchical) and horizontally (spatial). The complete implementation of this structure is encapsulated in the build_graph function, illustrated in Figure 24.

```
FUNCTION build_graph(item, tag2id, tfidf, scaler, comp_enc)
    root ← get_root_node(item)
    IF root IS null THEN
        RETURN null
    END IF
    nodes, edges ← traverse_dom(root)
    compute_subtree_stats(nodes)
    x ← build_features(nodes, tag2id, tfidf, scaler)
    IF LENGTH(edges) > 0 THEN
        edge_index ← TENSOR([[u, v] FOR EACH (u, v, _) IN edges], type=long).TRANSPOSE()
        edge_type  ← TENSOR([t FOR EACH (_, _, t) IN edges], type=long)
    ELSE
        edge_index ← EMPTY_TENSOR(shape=(2, 0), type=long)
        edge_type  ← EMPTY_TENSOR(shape=(0,), type=long)
    END IF
    y ← TENSOR(comp_enc.transform([item["label"]]), type=long)
    RETURN Data(
        x = x,
        edge_index = edge_index,
        edge_type = edge_type,
        y_component = y
    )
END FUNCTION
```

Figure 19. Build Graph Function

Normalization and Dataset Partitioning

To ensure numerical stability during training, continuous features (e.g., child count and depth) are standardized. This prevents features with larger magnitudes from disproportionately influencing the gradient descent process. The processed graphs are then partitioned into training, validation, and test sets using stratified sampling to maintain class balance, a process implemented in the split_dataset logic shown in Figure 25.

```
FUNCTION split_dataset(data, train_ratio = 0.7, val_ratio = 0.15, seed = 42)
    train_graphs, temp_graphs ← TRAIN_TEST_SPLIT(
        data,
        train_size = train_ratio,
        random_state = seed,
        shuffle = True
    )
    remaining_ratio ← 1.0 - train_ratio
    val_relative ← val_ratio / remaining_ratio
    val_graphs, test_graphs ← TRAIN_TEST_SPLIT(
        temp_graphs,
        train_size = val_relative,
        random_state = seed,
        shuffle = True
    )
    RETURN train_graphs, val_graphs, test_graphs
END FUNCTION
```

Figure 20. Dataset Splitting Function

Finally, the orchestration of the pre-processing and partitioning steps is managed by the system's entry point, illustrated in the Main Driver Function shown in Figure 26.

```
Algorithm: Dataset Preprocessing for Graph Construction

Input:
    JSON_PATH = source annotated dataset
    OUT_DIR = destination directory for processed outputs

Output:
    Train, validation, and test graph datasets
    Metadata file containing vocabulary and model input information

1. Create the output directory if it does not already exist.
2. Load the raw JSON dataset.
3. Build the tag vocabulary from the dataset and create a tag-to-index
mapping.
4. Initialize containers for:
   4.1 CSS class text corpus
   4.2 numeric node features
   4.3 component labels
   4.4 edge type counts
5. For each sample in the raw dataset:
   5.1 Extract the root DOM node.
   5.2 Skip the sample if no valid root exists.
   5.3 Store the component label.
   5.4 Traverse the DOM tree to obtain nodes and typed edges.
   5.5 Count the occurrence of each edge type.
   5.6 Compute subtree statistics for all nodes.
   5.7 Extract normalized CSS class strings for TF-IDF processing.
   5.8 Extract numeric structural features for each node.
6. Fit a TF-IDF vectorizer on the collected CSS corpus.
7. Fit a standard scaler on the numeric node features.
8. Fit a label encoder on the component labels.
9. Convert each raw sample into a graph using the learned mappings and feature
transformers.
10. Split the resulting graph dataset into training, validation, and test
subsets.
11. Save the three dataset splits.
12. Save metadata including:
    12.1 tag vocabulary
    12.2 class labels
    12.3 edge type mapping
    12.4 input feature dimension
```

Figure 21. Main Driver Function

Dataset Characteristics and Final Reporting

The pre-processing utility generates a comprehensive report summarizing the structural composition of the dataset. For the full inference training set (N=6,000), the

system maintains 500 samples per label across 96 feature dimensions. As detailed in the Final Pre-Processing Report seen in Tables Four, Five, and Six, this dataset contains approximately 38,000 hierarchical relationships and 25,000 sibling relationships. For the purposes of rapid iteration and hyperparameter optimization, a smaller subset (N=600) was also generated, the characteristics of which are captured in the Testing Graph Report as seen in Tables Seven, Eight, Nine

Metric	**Value (Subset)**	**Value (Full)**
Total Graphs	600	6000
Input Feature Dimension	96	96
Number of Classes	12	12

Table 6. Dataset Summary (Full Dataset)

Relationship	**ID**	**Count (Subset)**	**Count (Full)**
Parent-Child	0	1899	19,084
Child-Parent	1	1899	19,084
Sibling	2	2494	25,174

Table 7. Relationship Summary

Label	ID	Count (Inference)	Count (Training)
Card	0	500	50
DropdownMenu	1	500	50
Footer	2	500	50
Header	3	500	50
IconButton	4	500	50
ImageGallery	5	500	50
Link	6	500	50
NavigationBar	7	500	50
RangeSlider	8	500	50
SearchBar	9	500	50
Sidebar	10	500	50
Table	11	500	50

Table 8. Class Distribution

Semantic Bleaching for Comparative Analysis

To evaluate the impact of semantic information on model performance, a modified version of the pipeline was developed to perform "Semantic Bleaching." This process involves remapping HTML tags into generic categories and

stripping CSS attributes and TF-IDF vectors. As shown in Figure 29, the system utilizes a remapping array to collapse specific HTML elements into broader functional groups. By introducing depth jitter and tag bleaching—implemented via the modified traverse_dom function (Figure 30)—the feature input dimensions were reduced from 96 to 24. This leaves only the bare structural topology for neural network comparison, allowing for a controlled assessment of how much the model relies on raw structure versus semantic metadata.

```
# --- THE SEMANTIC BLEACH ---
# Map specific "giveaway" tags to generic containers.
TAG_REMAP = {
    # Structural giveaways -> generic div
    "nav": "div", "header": "div", "footer": "div", "section": "div",
    "article": "div", "aside": "div", "main": "div", "form": "div",

    # List/Table giveaways -> generic div
    "ul": "div", "ol": "div", "li": "div",
    "table": "div", "thead": "div", "tbody": "div", "tr": "div", "td": "div", "th": "div",

    # Text giveaways -> generic span
    "h1": "span", "h2": "span", "h3": "span", "h4": "span", "h5": "span", "h6": "span",
    "p": "span", "label": "span", "strong": "span", "em": "span", "i": "span", "b": "span",

    # WE KEEP FUNCTIONAL TAGS:
    # input, button, a, img, textarea, select
    # This allows models to distinguish "Interactive" vs "Static" content,
    # but removes the "Name" of the component.
}
```

Figure 22. Tag Remapping Array

```
FUNCTION ProcessNode(node, parent_idx, depth):
  GLOBAL nodes, edges

  # 1. Root Protection & Early Exit
  IF depth > 0 AND random_value() < 0.15: RETURN

  # 2. Tag Bleaching & Metadata
  raw = lower(node.get("tag") or node.get("type") or node.get("name") or "")
  tag = TAG_REMAP.get(raw, raw)
  attrs = copy(node.get("attributes", {})); attrs["class"] = ""

  # 3. Node Creation
  idx = length(nodes)
  curr_node = {
    "tag": tag, "attrs": attrs, "parent": parent_idx, "children": [],
    "depth": depth + random_uniform(-0.2, 0.2),
    "is_interactive": is_interactive(tag, attrs), "is_media": is_media(tag, attrs)
  }
  nodes.append(curr_node)

  # 4. Link to Original Parent
  IF parent_idx != None:
    edges.extend([(parent_idx, idx, 0), (idx, parent_idx, 1)])
    nodes[parent_idx]["children"].append(idx)

  # 5. Div Soup Injection (Conditional Wrapper)
  child_parent = idx
  IF random_value() < 0.10:
    wrap_idx = length(nodes)
    nodes.append({
      "tag": "div", "attrs": {}, "depth": curr_node["depth"],
      "parent": idx, "children": [], "is_interactive": 0, "is_media": 0
    })
    edges.extend([(idx, wrap_idx, 0), (wrap_idx, idx, 1)])
    nodes[idx]["children"].append(wrap_idx)
    child_parent = wrap_idx

  # 6. Recursion
  FOR child IN get_node_children(node):
    ProcessNode(child, child_parent, depth + 1)
```

Figure 23. Modified Traverse_Dom function with Semantic Bleaching Additions

GraphSAGE Training

```
CLASS HierarchicalGraphSAGE(NeuralNetworkModule):

    FUNCTION Initialize(in_dim, hidden_dim, out_dim):
        CALL super.Initialize()
        # Sequential SAGE layers with 'add' aggregation
        self.convs = [
            SAGEConv(in_dim, hidden_dim, aggr='add'),
            SAGEConv(hidden_dim, hidden_dim, aggr='add'),
            SAGEConv(hidden_dim, hidden_dim, aggr='add')
        ]
        self.post_nn = LinearLayer(hidden_dim, out_dim)

    FUNCTION ForwardPass(x, edge_index, batch_mapping):
        # Layer 1 & 2: Conv -> ReLU -> Dropout
        FOR i IN [0, 1]:
            x = self.convs[i](x, edge_index)
            x = Apply ReLU(x)
            x = Apply Dropout(x, p=0.3, training=self.training)

        # Layer 3: Final Conv without activation
        x = self.convs[2](x, edge_index)

        # Readout: Node-to-Graph aggregation
        x = GlobalMeanPool(x, batch_mapping)

        # Final Output Head
        x = Apply Dropout(x, p=0.3, training=self.training)
        RETURN self.post_nn(x)
```

Figure 24. Training Code Snippet: GraphSAGE Class

Classification and training were implemented using the PyTorch Geometric (PyG) library via a custom module, the

HierarchicalGraphSAGE class as seen in Figure 7, which facilitates learning from raw DOM graphs to predict component labels.

The network initialization constructs the computation graph using three stacked convolutional layers followed by a linear projection. To address the structural blindness of mean aggregation in shallow DOM trees, the convolutional layers are instantiated with aggr='add'. This forces the network to sum neighbor features rather than average them, preserving the specific count of child nodes.

The forward propagation logic, also detailed in Figure 7, defines the flow of data through the network. To introduce non-linearity, ReLU activation is applied between layers. Furthermore, to mitigate overfitting, a dropout of p = 0.3 is injected between the first and second layers; this randomly zeroes out elements of the feature vector during training, forcing the model to learn redundant representations of the DOM structure. The transition from node-level embeddings to graph-level outputs is handled by the global_mean_pool function, which utilizes a batch index

vector to average node embeddings specific to each graph instance, resulting in a fixed-size vector for every DOM tree in the batch.

Data Loading and Optimization Loop

```
# --- PHASE 1: DATA LOADING ---
PRINT "Loading datasets..."
train_ds, val_ds, test_ds = LOAD("train.pt"), LOAD("val.pt"), LOAD("test.pt")
meta = LOAD("meta.pt")

# Extract dimensions and classes
num_classes, in_dim = LENGTH(meta["classes"]), meta["input_dim"]
PRINT f"Input Dim: {in_dim} | Classes: {num_classes}"

# --- PHASE 2: INITIALIZATION ---
device = GET_COMPUTE_DEVICE()
model = HierarchicalGraphSAGE(in_dim, 64, num_classes).to(device)
optimizer = Adam(model.parameters, lr=1e-3, weight_decay=5e-4)

# Batching: 16 (Shuffle training only)
loaders = {
    "train": CreateDataLoader(train_ds, 16, True),
    "val":   CreateDataLoader(val_ds, 16, False),
    "test":  CreateDataLoader(test_ds, 16, False)
}

# --- PHASE 3: BALANCED LOSS ---
# Frequency-based weighting: N / (C * class_freq)
labels = [n.label FOR n IN train_ds]
counts = COUNT_FREQUENCIES(labels)
weights = LENGTH(labels) / (num_classes * (counts + 1e-6))
```

Figure 25. Data Loading and weighted loss calculation

To handle the unique topology of DOM trees, the implementation utilizes PyG's DataLoader, the logic of which

can be seen in Figure 25. Unlike standard tensor batching, the loader implements a diagonal stacking strategy where a batch of graphs is represented as a single, large, disjoint graph. The loader generates a batch vector—an index tensor mapping every node to its respective graph instance—allowing the GPU to process multiple independent nodes simultaneously without complex masking operations.

The training process is governed by a standard PyTorch optimization loop, as shown in Figure 33. Due to class imbalances in earlier datasets, inverse class weights were computed and passed to the Cross Entropy Loss function. This ensures that the gradient penalty for misclassifying underrepresented components remains high, even if the weights are less critical in the current balanced dataset.

Finally, the model's parameters are updated using the Adam optimizer with a learning rate of 0.001 and L2 weight decay over 100 epochs. In each step, the computation graph buffers are cleared via optimizer.zero_grad(), logits are computed, and backpropagation is performed via

loss.backward(). The results are then output as a classification report for final interpretation.

```
Algorithm: Training Loop and Validation

Input:
    model              = graph classification model
    training_loader    = mini-batch loader for training data
    validation_loader  = mini-batch loader for validation data
    criterion          = loss function
    optimizer          = optimization algorithm
    MODEL_SAVE_PATH    = destination for the best model checkpoint

Output:
    Trained model with the best validation F1-score saved

1. Initialize the best validation F1-score to zero.
2. For each epoch in the training schedule:
   2.1 Set the model to training mode.
   2.2 Initialize total training loss to zero.
   2.3 For each batch in the training loader:
       a. Move the batch to the selected device.
       b. Clear previous optimizer gradients.
       c. Perform a forward pass using node features, edge indices, and batch assignments.
       d. Compute the classification loss using the predicted outputs and true component labels.
       e. Perform backpropagation.
       f. Update the model parameters.
       g. Accumulate the batch loss.
   2.4 Set the model to evaluation mode.
   2.5 Initialize empty lists for predicted labels and true labels.
   2.6 Disable gradient computation.
   2.7 For each batch in the validation loader:
       a. Move the batch to the selected device.
       b. Perform a forward pass.
       c. Extract the predicted class labels.
       d. Store the predictions and true labels.
   2.8 Compute the macro F1-score from the collected validation predictions.
   2.9 If the current validation F1-score is better than the best recorded score:
       a. Update the best validation F1-score.
       b. Save the current model parameters.
   2.10 At regular reporting intervals, print the epoch number, average training loss, and validation F1-score.
3. End training and retain the best-performing model checkpoint.
```

Figure 26. Training Loop and Validation

System Inputs and the User Interface

The primary input of the study is a website design created by the user within a custom-built web environment. Using pre-defined blocks, users may lay out their website, add content, and set up basic triggers such as OnClick events. The current interface for this process, as hosted on Vercel, is shown in Figure 34. The editing environment features a dedicated playground and main editing area as shown in Figure 35, supplemented by a property editor and a live preview screen in Figure 36, which can also render the raw HTML output as seen in Figure 37.

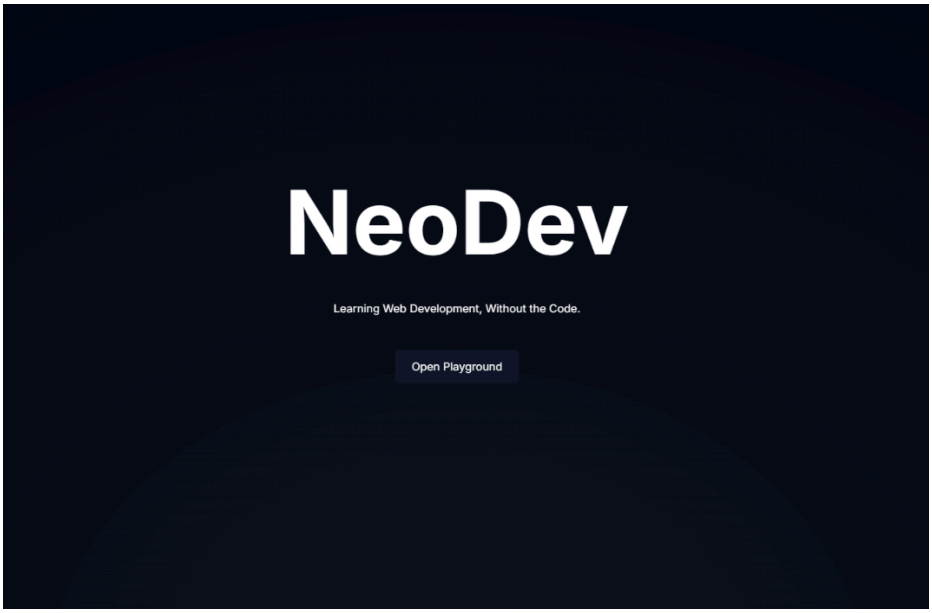

Figure 27. Web App Home Page as hosted on Vercel.

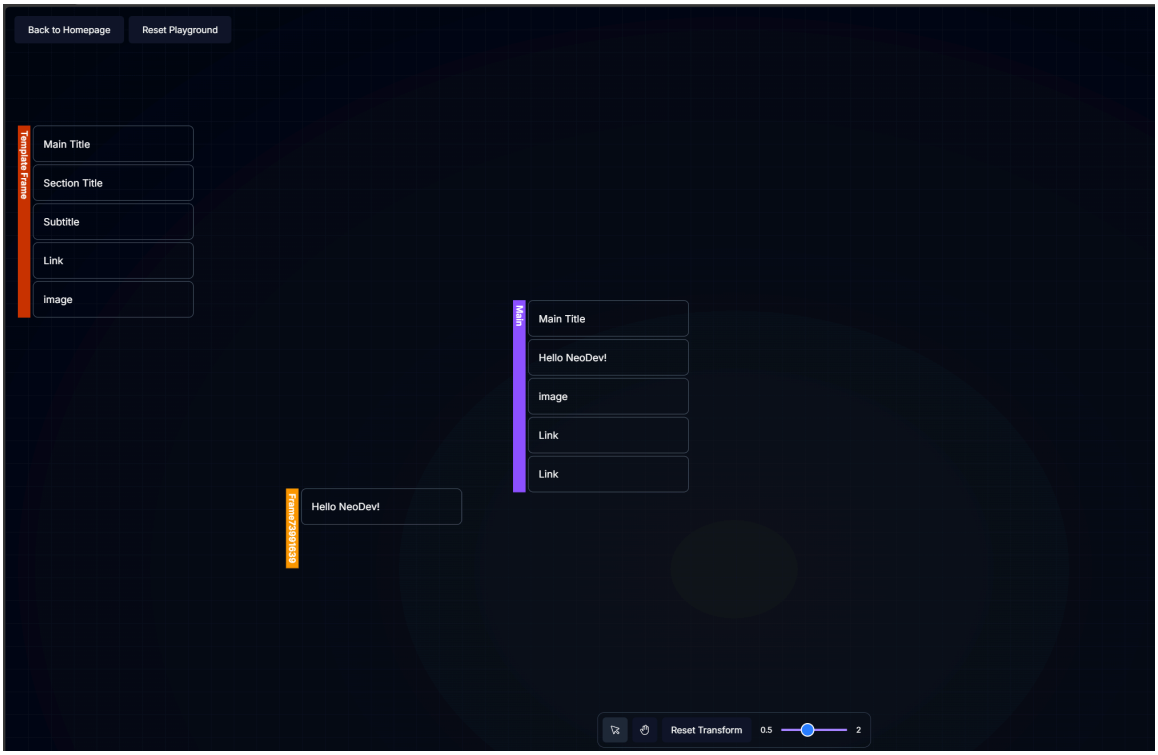

Figure 28. Current iteration of the Web App and editor, showcasing the playground and main editing areas

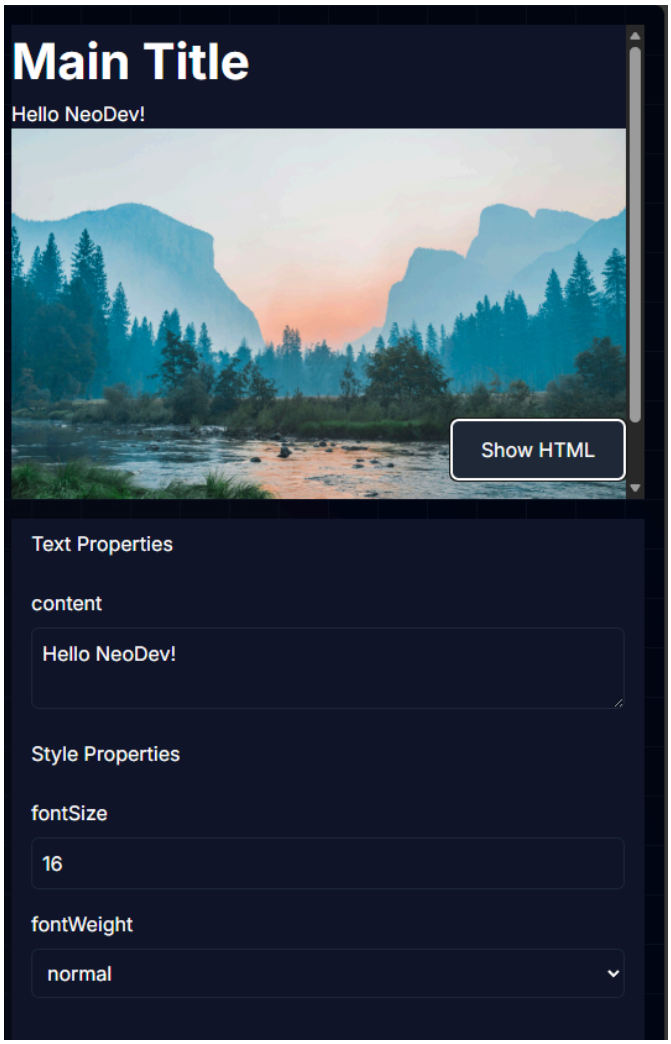

Figure 29. Current iteration of the Web App, showcasing the preview screen and property editors.

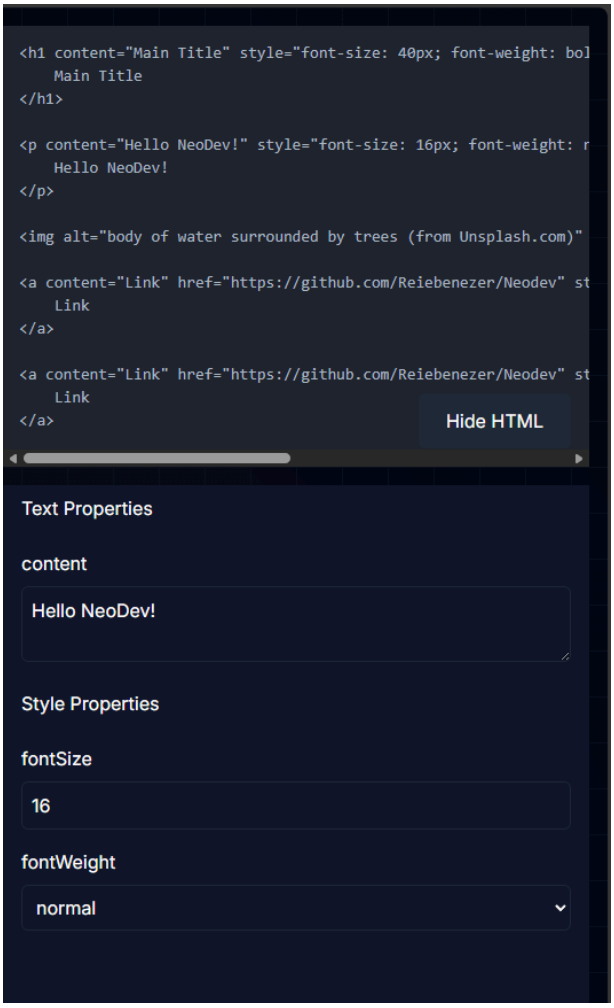

Figure 30. Same as above, but displaying the raw HTML Code

Data Pipeline and Model Integration

This design progress is saved to a JSON file, which represents the website as a hierarchical tree structure as seen in Figure 38. This file serves as the initial output of the project and is passed to the GraphSAGE classifier. To handle the raw DOM data, the system utilizes a preprocessor utility, as seen in Figure 40 to access DOM elements and prepare them for the graph-based architecture.

The model was initially trained using a dataset of scraped web UI components, a snippet of which can be seen Figure 39, ensuring the classifier was exposed to diverse real-world Document Object Model (DOM) structures. During the build phase, the JSON is passed to GraphSAGE again for optimizations and dependency applications before being sent to the code transpilers. These transpilers convert the JSON into a structured Frontend Framework file. The final outputs of this training and inference process include hierarchical classification results, confusion matrices, and an inference-ready model integrated with the web application.

```
import React from 'react';

export default function App() {
  return (
    <>
      <div style="display: flex; flex-direction: column; align-items: start; ">
        <div style="display: flex; flex-direction: row; align-items: center; ">
          <h1>MySimpleBlog</h1>
          <a>Home</a>
          <a>About</a>
          <a>Contact</a>
        </div>
      </div>
      <hr />
      <div style="display: flex; flex-direction: column; align-items: start; ">
        <h1>This is a heading</h1>
        <img src="https://images.unsplash.com/photo-1746730251885-34132bedecc5?
q=80&w=1472&auto=format&fit=crop&ixlib=rb-
4.1.0&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fH\nx8fGVufDB8fHw%3D" alt="Placeholder Image (from
Unsplash)" width="960" height="540" />
        <p>Lorem ipsum dolor sit, amet consectetur adipisicing elit. Aperiam enim architecto autem ea
accusamus unde, qui ducimus exercitationem natus, nulla odit et ipsum ul\nlam! Incidunt quos beatae
dolorum esse libero?</p>
      </div>
    </>
```

Figure 31. Initial JSON Output of website.

```
{
  "label": "Hero",
  "contents": [
    {
      "type": "div",
      "attributes": {
        "class": "relative isolate bg-white px-6 py-24 sm:py-32 lg:px-8"
      },
      "name": "relative isolate bg-white px-6 py-24 sm:py-32 lg:px-8",
      "children": [
        {
          "type": "div",
          "attributes": {
            "class": "absolute inset-x-0 -top-3 -z-10 transform-gpu overflow-hidden px-36 blur-3xl",
            "aria-hidden": "true"
          },
          "name": "absolute inset-x-0 -top-3 -z-10 transform-gpu overflow-hidden px-36 blur-3xl",
          "children": [
            {
```

Figure 32. Labeled_data.json dataset snippet

```
Algorithm: DOM Helper Functions and Preprocessor Initialization

1. Define a function to safely retrieve child nodes:
   1.1 Return the value of "children" if available.
   1.2 Otherwise, return the value of "childNodes" if available.
   1.3 If neither exists, return an empty list.

2. Define a function to construct a lightweight CSS/text token from a node:
   2.1 Retrieve the node attributes.
   2.2 Extract class, style, and name information.
   2.3 If the class value is a list, join it into a single string.
   2.4 Append available class, style, and name values into a token list.
   2.5 Join the token list into a single trimmed string and return it.

3. Define a function to extract the node tag:
   3.1 Read the node type or tag field.
   3.2 Convert the result to lowercase.
   3.3 If no valid tag is found, return "unknown".

4. Define a function to identify interactive tags:
   4.1 Return true if the tag belongs to {a, button, input, select, textarea}.

5. Define a function to identify media tags:
   5.1 Return true if the tag belongs to {img, video, audio, picture, svg, path}.

6. Initialize the preprocessor:
   6.1 Set the maximum number of CSS features.
   6.2 Set the random seed for reproducibility.
   6.3 Apply the seed to both the standard random generator and NumPy.
```

Figure 33. Preprocessor Code Snippet showing imports and DOM access

Results Interpretation and Analysis

Classifier Training and Hyperparameter Tuning

The training phase was executed in two distinct stages across four unique configurations. Initially, a pilot study was conducted using a reduced dataset (N=600), representing a 10% stratified sample of the total population. This stage served to calibrate the GraphSAGE hyperparameters and validate the structural integrity of the model architecture. Training during this pilot phase was performed on both the standard and "semantically bleached" variants of the data to establish a performance baseline. Following the optimization of the model parameters, the training was scaled to the full inference dataset (N=6,000). This comprehensive training pass utilized both data variants to evaluate the specific influence of semantic features (HTML tags and CSS classes) versus pure structural topology in the classification of UI components.

GraphSAGE Performance and Benchmarking

The classification performance of the GraphSAGE model was rigorously assessed through four training iterations, each designed to test a specific aspect of the model's predictive capability:

1. Standard Training (Inference): Performed on the full tailwind_augmented.json dataset to produce the final weights used for system inference.
2. Structural Benchmarking (Bleached): Conducted using the semantically bleached dataset to isolate the model's ability to identify components based solely on DOM connectivity and hierarchy.
3. Comparative Pilot: Executed on the tailwind_small.json dataset to facilitate direct performance comparisons with alternative architectures.

Dataset	Accuracy	Precision	Mac. Recall	Mac. F1	Wtd. Precision	Wtd. Recall	Wtd. F1	Samples
Semantically Bleached Small Dataset	0.88	0.9	0.89	0.9	0.89	0.88	0.88	91
Semantically Bleached Dataset	0.92	0.91	0.91	0.91	0.92	0.92	0.91	901
Small Inference Dataset	1	1	1	1	1	1	1	91
Inference Dataset	1	1	1	1	1	1	1	901

Table 9. Summary of Results across datasets

The resulting metrics as seen in Table 9, provide a summary of the model’s overall accuracy, precision, recall and F1-Scores for all datasets. Tables Nine and Ten, meanwhile, provide a granular view of the model's precision, recall, and F1-score across all twelve label categories. The high degree of accuracy observed in the standard dataset, relative to the bleached variant, highlights the critical role of stylistic markers in modern web component identification.

Label	precision	recall	f1-score	support
Card	1	1	1	8
DropdownMenu	1	1	1	3
Footer	1	1	1	7
Header	1	1	1	10
IconButton	1	1	1	7
ImageGallery	1	1	1	9
Link	1	1	1	11
NavigationBar	1	1	1	8
RangeSlider	1	1	1	4
SearchBar	1	1	1	7
Sidebar	1	1	1	11
Table	1	1	1	6

Table 10. GraphSAGE Classification Report - Testing Dataset

According to the classification reports in both Table 10 and Table 11, both the main inference model and the smaller stratified testing model were able to successfully predict all classes and classify all the labels correctly, without any errors or issues. Both the smaller and larger datasets were able to achieve a macro F1-Score of 1.00, which means that the classifier is working correctly, and accurately.

Label	precision	recall	f1-score	support
Card	1	1	1	85
DropdownMenu	1	1	1	83
Footer	1	1	1	65
Header	1	1	1	83
IconButton	1	1	1	60
ImageGallery	1	1	1	63
Link	1	1	1	76
NavigationBar	1	1	1	78
RangeSlider	1	1	1	78
SearchBar	1	1	1	76
Sidebar	1	1	1	81
Table	1	1	1	73

Table 11. GraphSAGE Training Results - Inference Dataset

As we can see in Table 11 above, upon the application of semantic bleaching, the model exhibited a marginal decrease in predictive accuracy, attributed to the removal of CSS-based stylistic markers and explicit HTML tag identifiers. Despite the absence of these semantic features, the GraphSAGE architecture maintained a high degree of structural awareness, achieving a macro F1-score of 91%. This demonstrates that the model is capable of high-fidelity classification based almost exclusively on the topological connectivity of the DOM as we can see in Tables 11 and 12

Label	precision	recall	f1-score	support
Card	1	1	1	85
DropdownMenu	1	1	1	83
Footer	0.42	0.17	0.24	65
Header	0.56	0.82	0.66	83
IconButton	1	1	1	60
ImageGallery	1	1	1	63
Link	1	1	1	76
NavigationBar	1	1	1	78
RangeSlider	1	1	1	78
SearchBar	0.99	1	0.99	76
Sidebar	1	0.99	0.99	81
Table	1	1	1	73

Table 12. Classification Report of Semantically Bleached Training

Label	precision	recall	f1-score	support
Card	1	1	1	8
DropdownMenu	1	1	1	3
Footer	0.38	0.43	0.4	7
Header	0.45	0.5	0.48	10
IconButton	1	1	1	7
ImageGallery	1	1	1	9
Link	1	1	1	11
NavigationBar	1	1	1	8
RangeSlider	1	1	1	4
SearchBar	1	0.86	0.92	7
Sidebar	1	0.91	0.95	11
Table	1	1	1	6

Table 13. Small Dataset Classification Report - Semantically bleached

While the majority of classes were successfully identified, the analysis revealed specific areas of structural ambiguity:

- Header and Footer Confusion: These two classes suffered the most significant performance degradation. From a purely topological perspective, both components often manifest as high-level container nodes with similar child-node distributions, leading to inter-class confusion when semantic labels are removed.

- Sidebar and Searchbar Sensitivity: A visible, albeit minimal, penalty was observed in the classification of Sidebars and Searchbars. The performance dip suggests that these components rely more heavily on specific semantic attributes (such as input types or navigation roles) to be distinguished from general container elements.

These results are also heavily reflected in the Confusion Matrices of the Model, particularly with the bleached dataset.

Figure 34 demonstrates the efficacy of the model in predicting the classes by showing which classes are confused with each other. As we can see, it is a straight diagonal line that matches the support level of the classification report. Additionally the zero values around this line indicate that all classes were successfully predicted.

Meanwhile in Figure 35, the confusion matrix for the semantically bleached dataset provides a clear indication of the model's operational limitations. Despite the removal of all semantic descriptors, ten of the twelve UI component

classes were correctly identified, affirming the GraphSAGE model's capacity for robust structural identification based solely on DOM connectivity.

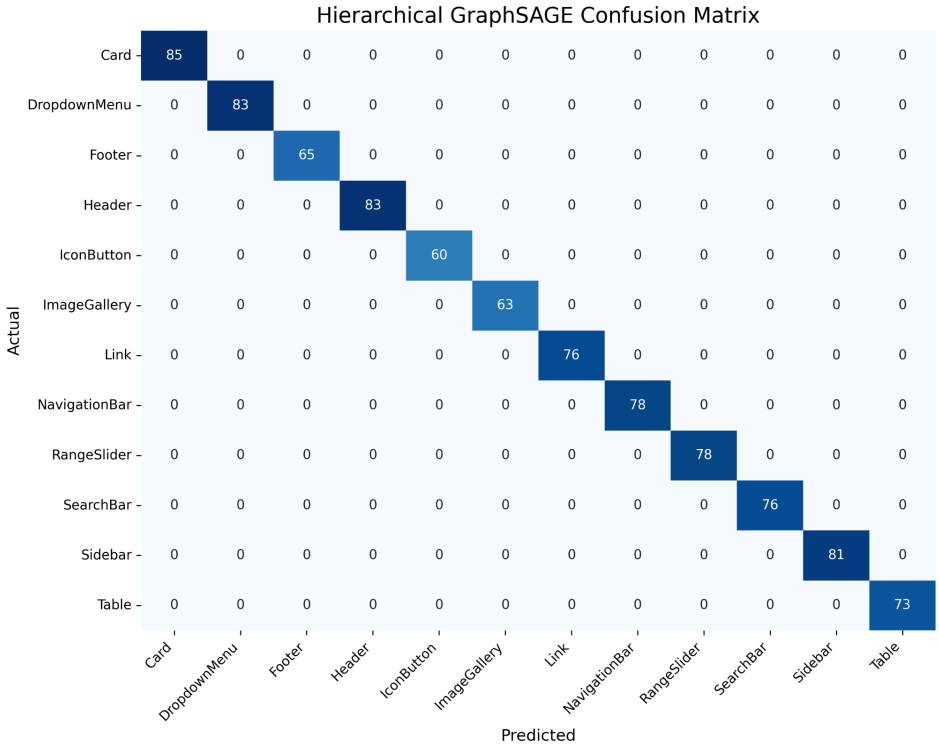

Figure 34. Confusion Matrix for GraphSAGE – Normal Dataset

However, the analysis revealed significant inter-class confusion between the Header and Footer components. This performance dip highlights the inherent topological similarities between these two classes; when stripped of identifying semantic features—such as tag names or CSS roles—their hierarchical structures and child-node

distributions are nearly indistinguishable to the model. This finding underscores the necessity of combining structural topology with semantic metadata for high-precision classification in complex web environments.

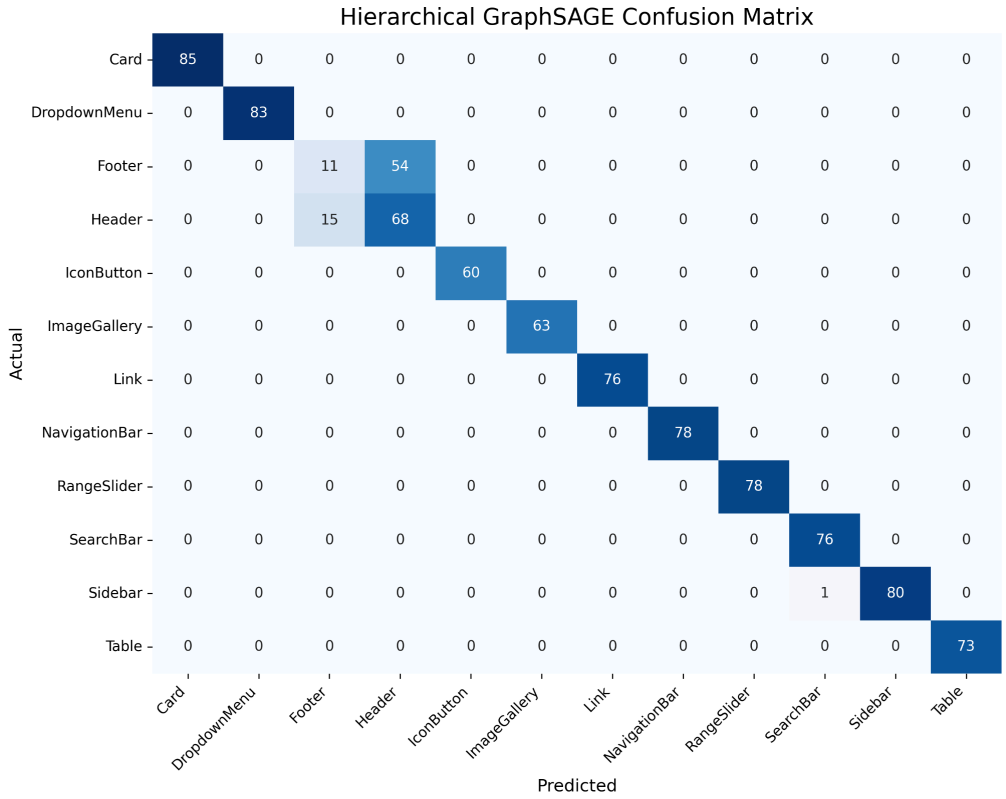

Figure 35. Confusion Matrix for GraphSAGE - Bleached

To validate the model's readiness for real-world application, further testing was conducted using a simulated inference payload. This procedure tested the classifier within an environment designed to emulate the production

requirements of the NeoDev system. This simulation evaluated the model's latency and predictive reliability when processing unobserved, live DOM structures, ensuring that the classification pipeline remains performant under operational conditions as we can see in Table 13, which shows the results of the API when tested locally.

Test	Status	Testing Dataset Prediction	Correct?	Testing Dataset Confidence	Full Dataset Prediction	Correct?	Full Dataset Confidence
SearchBar	200 OK	SearchBar	Yes	82.18%	SearchBar	Yes	95.06%
Header	200 OK	Navigation Bar	No	85.75%	Navigation Bar	No	99.99%
Footer	200 OK	Sidebar	No	56.19%	Navigation Bar	No	95.04%
Card	200 OK	Card	Yes	95.11%	Card	Yes	99.37%
Table	200 OK	Table	Yes	52.86%	Table	Yes	90.83%

Table 14. Inference Testing using Graphsage with full and training dataset

In Table 13, we can see that the Inference results show the classifier was able to predict 3 out of 5 payloads with accuracies above 90%, however, even though the model was able to successfully classify Headers and Footers during

training and testing, It can be seen that it still struggles to do so in a production environment, demonstrating the limitations of the GNN.

Additionally, we can also see in Table 13 that it holds true for the smaller testing dataset, albeit with a reduction in confidence due to the smaller number of samples.

Comparison with Other Neural Networks

For comparison, two Neural Networks were selected, a basic Convolutional Neural Network that treats each graph as an image, as well as another Graph Neural Network architecture, GraphCONV or GCN, a basic GNN introduced in 2017, and is a generalization of CNNs to graph data. Training was conducted using the same settings as GraphSAGE.

There were no changes to pre-processing and semantic bleaching steps. Only the smaller dataset, tailwind_small.json was used for training and testing these models due to their smaller size.

Graph Convolutional Network

Label	precision	recall	f1-score	support
Card	1	1	1	8
DropdownMenu	1	1	1	3
Footer	1	1	1	7
Header	1	1	1	10
IconButton	1	1	1	7
ImageGallery	1	1	1	9
Link	1	1	1	11
NavigationBar	1	1	1	8
RangeSlider	1	1	1	4
SearchBar	1	1	1	7
Sidebar	1	1	1	11
Table	1	1	1	6
accuracy			1	91
macro avg	1	1	1	91
weighted avg	1	1	1	91

Table 15. GCN Classification Report - Normal Dataset

Using the classification report as seen in Table 15, it can be seen that the GCN was able to achieve the same level of accuracy and performance as GraphSAGE. The GCN managed to classify and predict all the labels correctly, achieving a macro F1 score of 1.00. A score similar to what the GraphSAGE model was able to achieve.

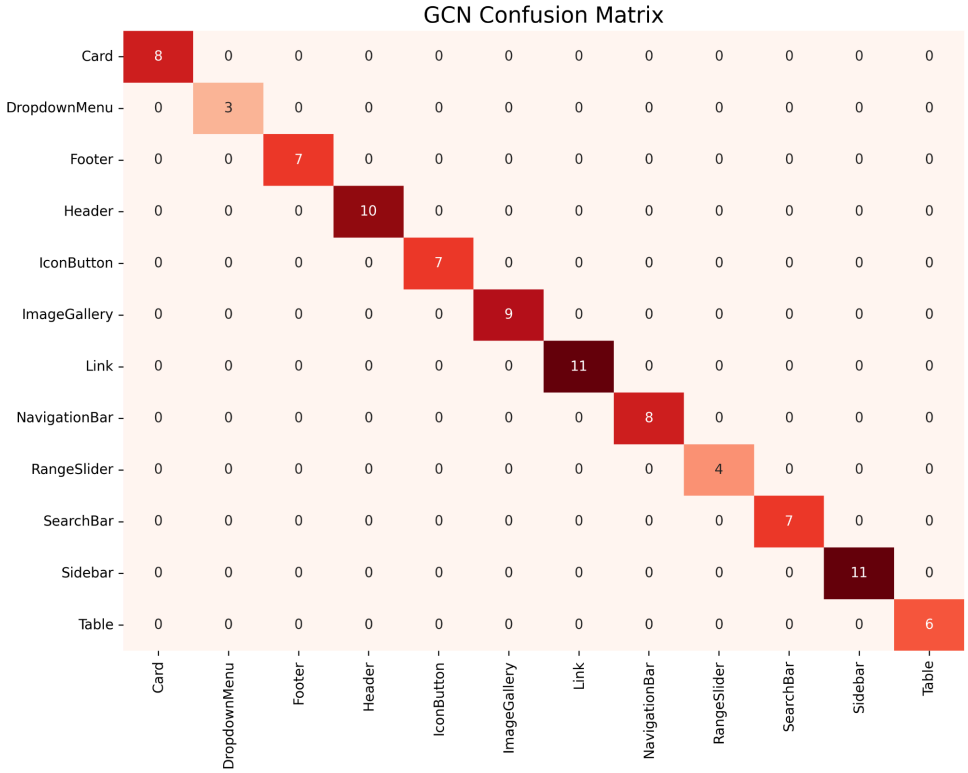

Figure 36. GCN Confusion Matrix

Similarly, the Confusion Matrix as shown in Figure 36 for the GCN also tells the same story. The Neural Network's predicted classes all lined up well with that of the actual class, indicating that the GCN managed to correctly predict all the labels it was tested on.

Label	precision	recall	f1-score	support
Card	1	1	1	8
DropdownMenu	1	1	1	3
Footer	0.36	0.71	0.48	7
Header	0.6	0.3	0.4	10
IconButton	1	1	1	7
ImageGallery	1	1	1	9
Link	1	1	1	11
NavigationBar	1	1	1	8
RangeSlider	1	1	1	4
SearchBar	1	0.86	0.92	7
Sidebar	1	0.91	0.95	11
Table	1	1	1	6
accuracy			0.88	91
macro avg	0.91	0.9	0.9	91
weighted avg	0.91	0.88	0.88	91

Table 16. GCN Bleached Results

Again, when looking at the semantically bleached results in Table 16, we can see that the GCN struggled in a number of labels. Footer and Header continue to be a major classification issue, achieving mediocre precision and recall. Additionally, we can see some issues with searchbar and sidebar classes, likely due to their similarity with other structures.

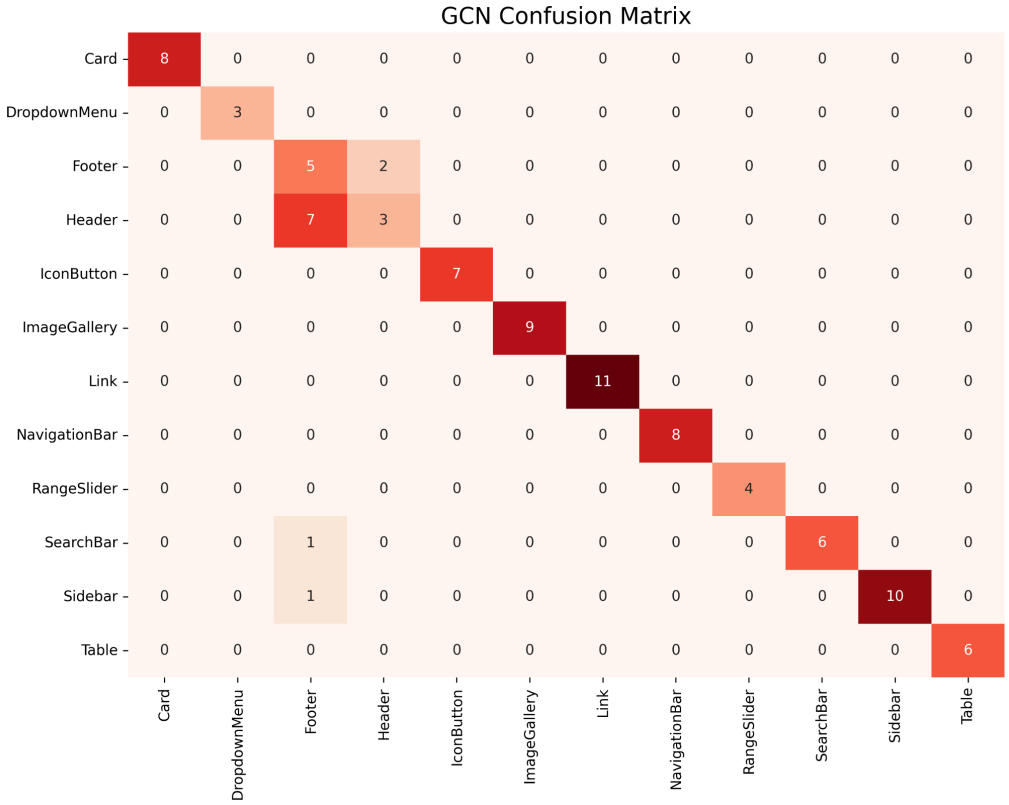

Figure 37. GCN Bleached Confusion Matrix

Additionally, the confusion matrix for the bleached Dataset as in Figure 37 reveals further information as to which labels were mistaken for each other. In the case of headers, 2 were identified as footers while the three were successfully labeled. Similarly, five footers were accurately identified, but seven footers were identified as headers instead. We can also see that some footers were mistakenly identified as Searchbar and Sidebar respectively.

Convolutional Neural Network

Label	precision	recall	f1-score	support
Card	1	1	1	8
DropdownMenu	1	1	1	3
Footer	1	1	1	7
Header	1	1	1	10
IconButton	1	1	1	7
ImageGallery	1	1	1	9
Link	1	1	1	11
NavigationBar	1	1	1	8
RangeSlider	1	1	1	4
SearchBar	1	1	1	7
Sidebar	1	1	1	11
Table	1	1	1	6
accuracy			1	91
macro avg	1	1	1	91
weighted avg	1	1	1	91

Table 17. CNN Results - Standard dataset.

While GNNs theoretically offer superior structural modelling of UI components, they were able to achieve similar performance with GraphSAGE and GraphConv, achieving a Macro F1-Score 1.00, as can be found in Table 17. Indicating that there is no significant difference among models. This also suggests that the serialized sequence of functional tags contain sufficient structural signals for

classification, rendering the computational overhead of a graph-based message passing unnecessary for these tasks.

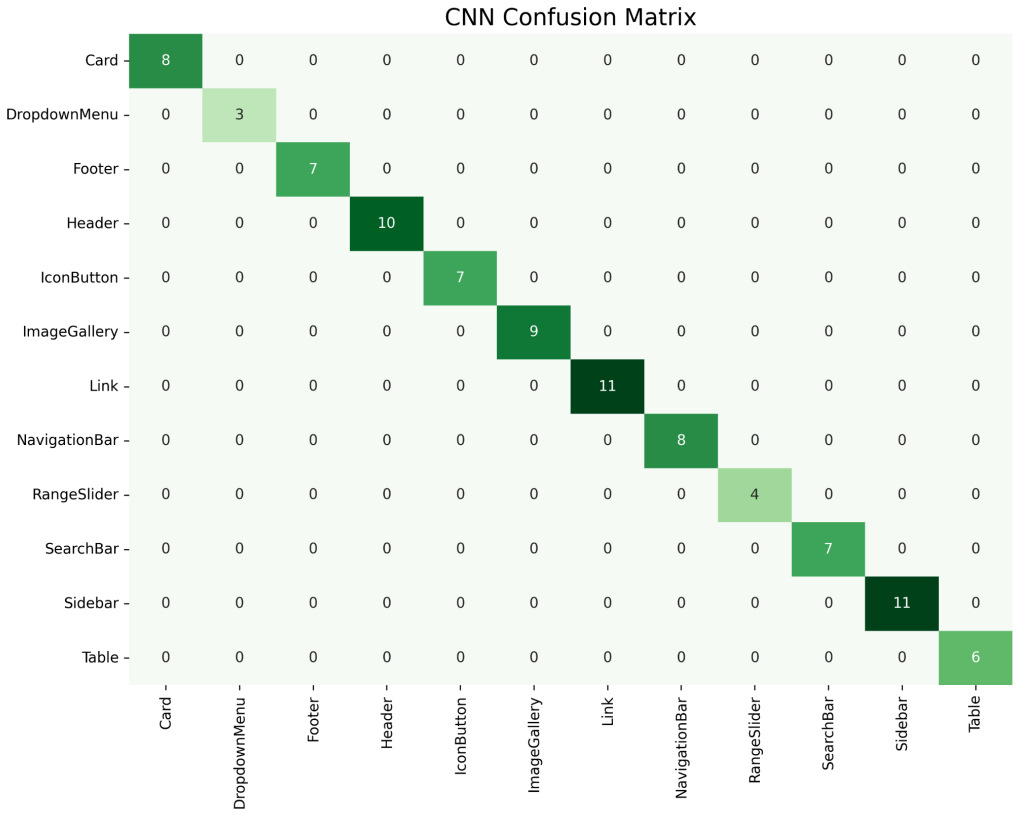

Figure 38. CNN Confusion Matrix

The confusion matrix as demonstrated in Figure 38 also corroborates this by demonstrating that the all predicted labels matched the actual labels. However, as with earlier, Semantic Bleaching and artificial data scarcity is where we can see the slight differences between the models

Label	precision	recall	f1-score	support
Card	1	1	1	8
DropdownMenu	1	1	1	3
Footer	0.36	0.57	0.44	7
Header	0.38	0.3	0.33	10
IconButton	1	1	1	7
ImageGallery	1	1	1	9
Link	1	1	1	11
NavigationBar	1	1	1	8
RangeSlider	1	1	1	4
SearchBar	1	0.86	0.92	7
Sidebar	1	0.91	0.95	11
Table	1	1	1	6
accuracy			0.87	91
macro avg	0.89	0.89	0.89	91
weighted avg	0.88	0.87	0.87	91

Table 18. CNN Classification Report - Bleached Dataset

With a Macro F1-score of 0.87, the CNN's lack of ability to understand structures and trees become visible, especially when compared to GCN and GraphSAGE's scores of 0.90 as shown in Table 18. This indicates that the hypothesis of CNNs struggling with structures and trees still holds true, however, the accuracy remains perfectly acceptable nonetheless. Additionally, it's worth noting that the dataset's graphs are noted to be broad but shallow,

having only 2-3 nodes per graph. This shallowness is what allows the models to achieve structural saturation, allowing them to see the entire graph at once, and rending the advantages of the GNNs moot

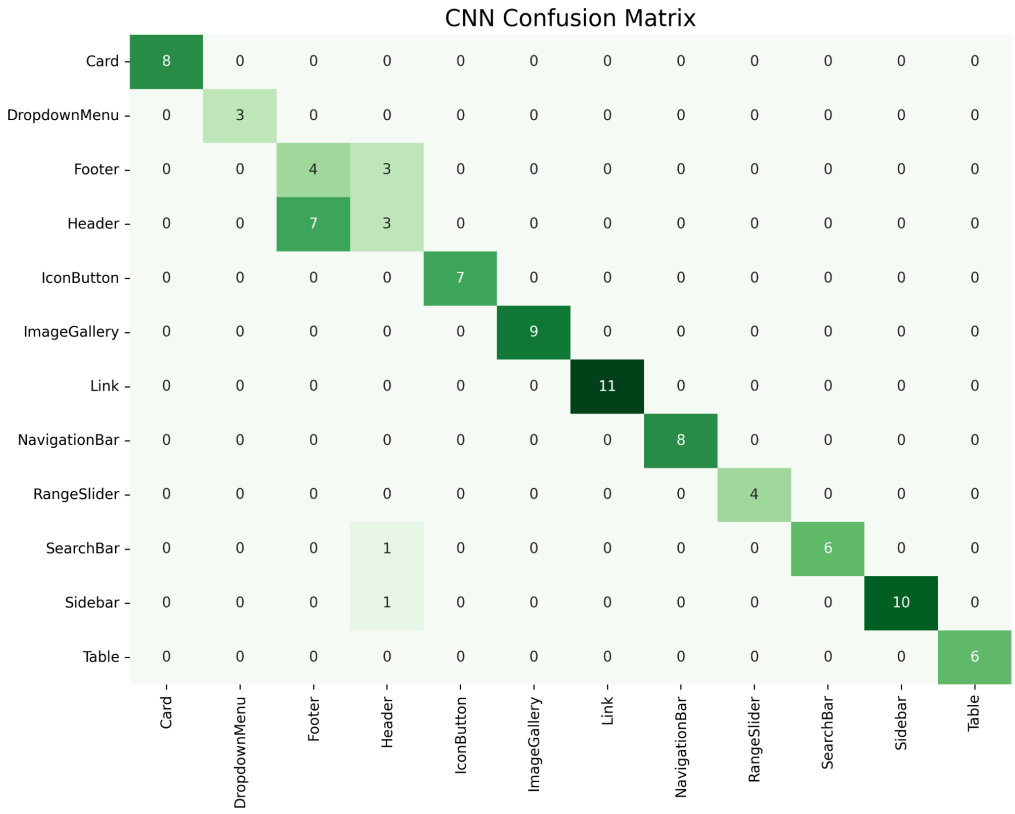

Figure 39. CNN Confusion Matrix – Bleached Dataset

As can be seen in Figure 39, the CNN's confusion matrix also further corroborates this by demonstrating the same patterns with the other architectures. Headers and Footers

still remain a significant source of confusion and error in an otherwise accurate classifier. In order to truly demonstrate the effectiveness of GNNs in component classification, more complex components must be tested to see the difference.

Summary:

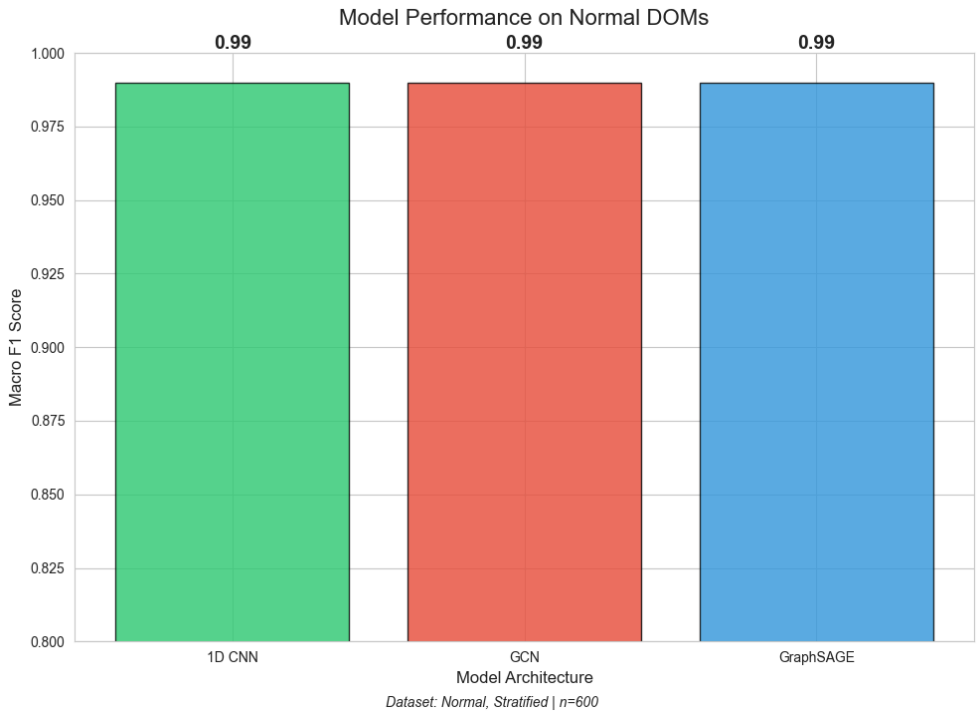

Figure 40. Model Comparison Bar Chart

Overall, all three models achieved a significant performance parity compared to each other, especially in normal classification as can be found in Figure 57, due to the shallowness of the graph allowing the CNN to catch up with the GNNs. However, when the dataset is semantically bleached, CNN's flaws can be clearly seen.

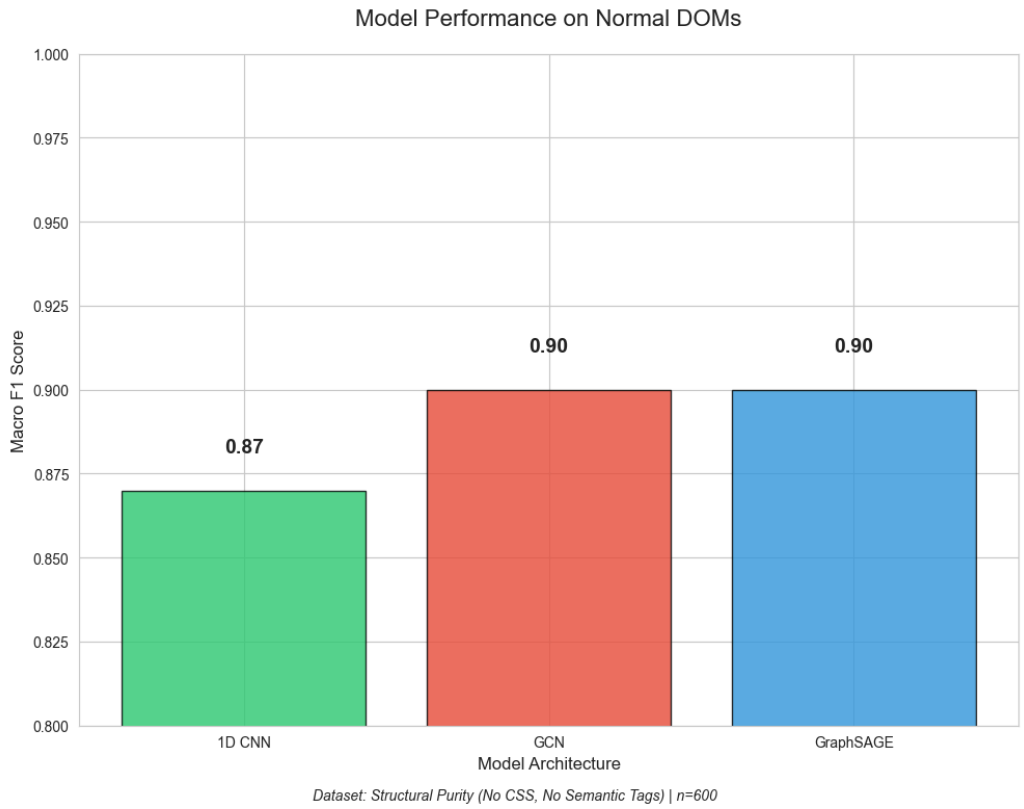

Figure 41. Model Comparison Chart - Semantically bleached

Figure 58 shows that with semantic bleaching, only the structure of the DOM remains as a feature, and while this means that all models suffered a hit to their performance, both GCN and GraphSAGE still managed to achieve a macro F1-score 0.90 while CNNs struggled, despite being able to see the entire graph as they cannot easily understand trees and structures.

Transpiler Accuracy and Speed

The accuracy of the transpilation process was assessed through the structural integrity of the generated frontend framework code. Because the system utilizes a rules-based mapping, where JSON nodes are directly translated into predefined React or Svelte components, the logic-to-code mapping achieved high fidelity. Expert evaluators noted that the system produced valid Svelte and React structures, with components nested correctly according to the user's design. The transpiler successfully preserved attributes like image sources, class lists, and inline styles, ensuring that the visual output in the "Live Preview" matched the exported code. Furthermore, the system successfully preserves attributes such as Tailwind classes and ARIA labels, ensuring the exported code adheres to the accessibility standards.

To optimize performance and modularity, the transpiler is hosted as a dedicated web service using Python and FastAPI. This architecture allows the main IDE to send a POST request containing the website's JSON body to the API, which then returns the transpiled code.

- Latency: The end-to-end request-response cycle (JSON input to Code output) typically executes in under 150ms for standard UI components.
- Efficiency: Because the rules-based conversion follows a linear O(n) traversal of the DOM tree, the computational overhead remains minimal, providing a near-instantaneous "Export" experience for the user.

Comparison with existing applications

As per objective four, NeoDev's features and capabilities were also compared with other similar applications. These included Builder.io, a specialized site builder with visual editing properties and the option to import Figma files, FigmatoCode, a specialized plugin that converts figma files into useable framework-ready code, Webflow, a wordpress like website that can be used to create websites, and finally scratch. The similarities and differences in terms of features and capabilities are summarized in the following page in Tables 19 and 20.

Platform	Input Method	Output	Primary Purpose	Target Audience	Educational Focus	Code Export
NeoDev	Block-based drag-drop (JSON hierarchy)	React, Svelte framework code	Web development learning tool	Students, educators	High (primary goal)	Yes (framework code)
Builder.io	Figma design files, Visual editor, Text prompts	React, Vue, Angular, Next.js, Svelte, Qwik, Solid, HTML	Production website development	Professional developers, design teams	Low (production focus)	Yes (framework code)
FigmaToCode	Figma design files (plugin)	HTML, React, Svelte, Tailwind, Flutter, SwiftUI	Design-to-code conversion	Developers, designers	Moderate (transparent algorithms)	Yes (7 frameworks)
Webflow	Visual Designer (GUI)	HTML, CSS, JavaScript (static)	Professional web design	Designers, agencies	Low (design focus)	Yes (HTML/CSS/JS)
Scratch	Block-based drag-drop (visual blocks)	Scratch programs (no web code)	Programming education	Children ages 8-16	High (primary goal)	No (platform-specific)

Table 19. Platform Comparison Matrix

Feature	NeoDev	Builder.io	FigmaToCode	Webflow	Scratch	W3Schools
Real-time Preview	Yes (Live Preview)	Yes	No (plugin-based)	Yes	Yes (Stage)	Yes (Try It Editor)
AI/ML Classification	Yes (GraphSAGE hierarchical)	Yes (proprietary)	No (rules-based layout analysis)	No	No	No
Framework Support	2 (React, Svelte)	8+ frameworks	7 (HTML, React, Svelte, Tailwind, Flutter, SwiftUI, styled-components)	1 (HTML/CSS/JS)	N/A (Scratch only)	N/A (direct coding)
Transpilation Approach	Rules-based deterministic	AI-assisted generative	Rules-based with intermediate representation	Real-time generation	N/A	N/A
Block-Based Interface	Yes (HTML/CSS blocks)	No (visual/text prompts)	No (Figma input)	No (visual designer)	Yes (programming blocks)	No (text editor)
Deployment	Vercel (free tier)	Builder Cloud	User-managed (exported code)	Webflow hosting	MIT servers	W3Schools Spaces

Table 20. Technical Capabilities Comparison

Comparative Analysis

Comparison Limitations, Direct performance benchmarking with commercial platforms like Builder.io was not feasible for this study. Builder.io's proprietary algorithms are closed-source, and the platform does not provide public API access for controlled testing. Additionally, as noted in Tables 18 and 19 NeoDev and Builder.io operate on fundamentally different input formats. NeoDev processes JSON block hierarchies created through drag-and-drop interactions, as shown in Figures 16-17. Builder.io converts Figma design files or uses a visual GUI editor with large language models and agentic AI. This architectural difference makes direct performance comparison scientifically invalid because the two systems solve different problems with different starting points.

Processing speed cannot be meaningfully compared because NeoDev's transpilation begins with a JSON block structure while Builder.io begins with a Figma design file. The computational overhead of parsing these different input formats varies significantly. Any speed comparison would depend on format conversion time rather than actual

algorithmic efficiency. A benchmark test in NeoDev measures the time from block-to-code conversion, while the equivalent Builder.io test would measure time from design import to code generation. These are not equivalent measurements of system performance.

Classification accuracy presents another challenge. NeoDev's GraphSAGE model achieves a 93.54 percent element-level F1 score on the dataset described in Chapter 3. However, Builder.io does not disclose what it classifies, how accuracy is measured, or what benchmark datasets contain. Their AI is described only as proprietary machine learning in marketing materials. Without knowledge of Builder.io's classification task, target labels, or evaluation methodology, claiming that one system is more accurate than the other would be scientifically meaningless. The classification problems are fundamentally different. NeoDev classifies DOM structure(Document Object Model) and identifies component patterns within a block-based representation. Builder.io, if it classifies at all, must infer semantic structure from visual design elements.

Code quality represents another subjective dimension that resists objective measurement. NeoDev generates semantic HTML with descriptive class names and unminified output optimized for educational readability, as documented in Chapter 4. Builder.io generates code optimized for professional production environments with contextual class names that align to existing design systems and coding conventions. The platform emphasizes clean, contextual class names and CSS variable names aligned to design systems rather than minimal abbreviations. Builder.io's code generation can be customized through Custom Instructions to match specific team coding standards, framework preferences, and project requirements. Determining which approach produces better code depends entirely on the use case. Learning environments favor readability so students can study the code, while production deployments favor integration with existing systems and team conventions. No universal quality metric exists that accounts for these different objectives because the evaluation criteria themselves are use-case specific.

Output code size varies dramatically based on component complexity, styling choices, and framework conventions. A simple button component might generate 50 lines in one framework and 30 lines in another, not because of algorithmic superiority but due to framework-specific syntax requirements. JavaScript frameworks like React require component wrapping and may include additional metadata, while vanilla HTML requires only basic structure. Without a standardized test suite of identical UI components that both platforms could process from a neutral starting point, code size comparisons would reflect framework differences rather than platform capabilities.

Builder.io does not publish performance metrics, algorithm descriptions, or evaluation results in their technical documentation. Their blog contains marketing case studies and feature announcements but no peer-reviewed performance analysis. This lack of transparency makes quantitative comparison impossible without fabricating estimates, which would violate research integrity standards.

Architectural Differences Between Platforms, Understanding why platforms cannot be directly compared

requires examining their fundamental architectural differences, which reflect divergent design philosophies and target audiences. These architectural choices determine what each platform optimizes for and what trade-offs it accepts.

NeoDev accepts JSON block hierarchies as input, generated through the drag-and-drop block editor shown in Figures 16-17. Each block represents an HTML element with attached style properties, creating a structured tree that explicitly encodes the separation of concerns between HTML structure and CSS styling. This format is pedagogically transparent. Students can see exactly how HTML elements nest and how styles apply to specific elements. The block-based input ensures that the underlying DOM structure is always valid and complete before transpilation begins.

Builder.io, in contrast, accepts Figma design files or uses a visual WYSIWYG editor. Figma designs contain visual layers with absolute positioning, colors, and typography but do not encode semantic HTML structure. Builder.io's AI must infer appropriate HTML tags from the visual design, deduce component boundaries based on position and styling, and convert visual positioning into CSS layout systems like

flexbox or grid. These inference tasks are computationally complex and error-prone, requiring sophisticated computer vision and semantic understanding techniques.

FigmaToCode operates similarly to Builder.io in accepting Figma design files as input but differs fundamentally in approach and purpose. FigmaToCode is an open-source Figma plugin with over 4,500 stars on GitHub that converts Figma designs into responsive layouts using a sophisticated multi-step process. The plugin first converts Figma's native nodes into JSON representations, preserving all necessary properties while adding optimizations and parent references. These JSON nodes are then transformed into AltNodes, a custom virtual representation that can be manipulated without affecting the original design. The plugin analyzes and optimizes layouts, detecting patterns like auto-layouts, responsive constraints, and color variables. Finally, the optimized structure is transformed into the target framework's code with special handling for each framework's unique patterns and best practices. FigmaToCode supports HTML, React (JSX), Svelte, styled-components, Tailwind, Flutter, and SwiftUI code

generation. The plugin handles complex edge cases including mixed positioning with absolute and auto-layout, color variables for theme-consistent output, and gradients and effects that require specialized conversion logic. Unlike Builder.io's proprietary AI approach, FigmaToCode uses deterministic rules-based conversion with an intermediate representation that allows for sophisticated transformations before code generation. The open-source nature means developers can inspect, modify, and extend the conversion logic, making it valuable for understanding design-to-code algorithms.

Webflow operates similarly to Builder.io in accepting visual designs through its Designer interface but differs in approach. Webflow uses a visual designer where users manipulate elements directly on a canvas, and the platform generates code in real time as design changes occur. Users on paid Workspace Plans can export clean, static HTML, CSS, JavaScript, and assets. The exported code is described as clean, semantic, and well-structured, allowing developers to edit it in any code editor. However, exported sites lose dynamic features like CMS content, eCommerce functionality,

and forms because these require Webflow's backend infrastructure. Webflow's code export is designed for delivering completed projects to clients or hosting on alternative platforms without Webflow attribution.

Scratch represents a fundamentally different category. Developed by MIT in 2004, Scratch is a block-based programming language designed for ages 8-16 to teach programming concepts without textual coding. Users drag and drop color-coded blocks representing programming constructs like loops, conditionals, and events to create games, animations, and interactive stories. Scratch prevents syntax errors entirely because blocks fit together like puzzle pieces, only connecting when logically valid. The platform abstracts away implementation details to focus on computational thinking and problem-solving skills. Scratch does not generate web code. Instead, it executes programs within its own runtime environment hosted on MIT servers. The educational philosophy prioritizes concept mastery over production output.

W3Schools occupies yet another position in the learning ecosystem. Founded in 1998, W3Schools provides free online

tutorials, references, and exercises covering HTML, CSS, JavaScript, Python, SQL, Java, and many other technologies. The platform emphasizes syntax learning through detailed explanations and code examples. Users write code directly in text editors provided by the platform. The Try It Yourself editor allows students to edit examples and run code directly in the browser, seeing results in real time. W3Schools also offers Spaces, which enables students to create and host live websites using HTML, CSS, and JavaScript, as well as full-stack server environments with PHP, Node.js, Java, Python, C#, and Rust. Teachers can access dashboards to manage student groups and track progress. The platform serves over 70 million monthly visitors and has been operating for more than 25 years. W3Schools focuses on syntax mastery and hands-on practice rather than visual abstraction or design-to-code conversion.

The classification purposes also diverge significantly across platforms. NeoDev's GraphSAGE classifier identifies both component types, such as navigation bars, hero sections, and cards, and individual HTML element types like div, span, and button within each component, as demonstrated

in the hierarchical classification results shown in Figures 41 and 42. This dual-level classification serves an educational purpose by providing students with insights about web component patterns and structural composition. The system can explain that a structure looks like a navigation bar, which typically contains list elements and anchor tags for links, as documented in the GraphSAGE Pipeline described in Figure 6.

Builder.io's classification, if it occurs at all, serves a production purpose: converting visual designs into functional code as quickly as possible. The company does not disclose whether they classify components, what taxonomy they use, or whether classification occurs versus direct visual-to-code conversion. FigmaToCode does not employ machine learning classification. Instead, it uses deterministic layout analysis to detect patterns like auto-layouts and responsive constraints during the intermediate representation phase. The plugin analyzes parent-child relationships and z-index ordering to produce accurate representations, but this analysis follows algorithmic rules rather than trained classification models.

Webflow does not employ classification because users build directly in a visual environment where element types are explicitly selected. Scratch does not classify web components because it operates in a different domain focused on general programming concepts. W3Schools does not classify user code because it provides tutorials and editors for direct text-based coding.

Transpilation approaches reflect these different philosophies. NeoDev employs a rules-based transpiler described in Chapter 3. The system applies deterministic transformations where HTML tags map to JSX syntax for React or standard HTML for Svelte, CSS properties are extracted and deduplicated by the StyleManager using MD5 hashing, and framework-specific syntax adjustments occur through predefined templates. This approach guarantees consistent, predictable output that students can study and understand. The rules are explicit and can be inspected, modified, and extended for educational purposes.

Builder.io uses AI-assisted generation where machine learning models generate code based on patterns learned from training data. This approach can adapt to design intent and

produce sophisticated layouts that account for visual relationships and design context. However, the generation process operates as a black box that provides no transparency into why particular code was generated or how to modify the generation rules. Students cannot examine and learn from the transpilation process because it involves neural network inference rather than interpretable rules.

FigmaToCode uses a rules-based approach similar to NeoDev but operates on Figma design files rather than blocks. The plugin's conversion process is deterministic and follows explicit transformation rules organized in a monorepo architecture with separate packages for backend logic and UI components. The backend package contains the business logic that reads the Figma API and converts nodes, while framework-specific code generation handles unique patterns and best practices for each target language. Because the project is open source, developers can inspect the conversion algorithms, understand the decision logic, and modify transformation rules to suit specific needs. This transparency makes FigmaToCode valuable for educational purposes and algorithm research, unlike proprietary AI-based

tools. The plugin uses esbuild, a high-speed bundler for CSS and JavaScript code for fast compilation and Turborepo for managing the monorepo, with only modified files recompiled when changes are made.

Webflow generates code in real time as users manipulate elements in the Designer. The exported code follows standard HTML, CSS, and JavaScript conventions and is described as optimized, semantic, and well-structured. Webflow generates class names automatically based on element naming in the Designer. The platform uses Normalize.css and includes JavaScript for interactions and animations. Because code generation is tied directly to visual manipulation, the output reflects user design choices rather than AI interpretation.

Scratch does not transpile to web technologies. Programs created in Scratch execute within the Scratch runtime environment using its proprietary block-based language. Users can share projects on the Scratch website where others can view and remix them, but there is no code export to HTML, CSS, or JavaScript. W3Schools does not perform transpilation because users write code directly in

the target language. The platform provides syntax examples and interactive editors but does not convert between formats.

Target audiences determine optimization priorities across all platforms. NeoDev targets students and educators in academic environments, as specified in Chapter 1. The platform prioritizes learning outcomes where code must be readable, structure must be explicit, and the relationship between blocks and generated code must be transparent so students can learn from studying the output. Performance optimization and production features are secondary considerations. An educational system should generate code that teaches students about web development, not code optimized for deployment bandwidth.

Builder.io targets professional developers and design teams building commercial websites. The platform prioritizes deployment readiness where code must integrate with existing codebases, workflows must be efficient for rapid development, and output must follow team conventions and design system alignment. Educational value is not a design

consideration because users are professionals who already understand web development.

FigmaToCode targets developers and designers who want free, open-source conversion from Figma to code without subscription costs. Target users include developers who want to understand design-to-code algorithms, teams that need customizable conversion logic, and individuals seeking alternatives to paid commercial tools. The open-source nature allows users to modify conversion rules, add support for new frameworks, or fix bugs independently without vendor dependencies.

Webflow targets designers and agencies creating professional websites. The platform emphasizes visual design control and eliminates coding barriers for designers who want pixel-perfect implementations without writing code manually. Webflow users range from complete beginners creating simple sites to professional agencies managing client projects. The code export feature serves professionals who need to deliver projects to clients or host sites independently.

Scratch targets children ages 8-16 learning fundamental programming concepts. The platform prioritizes computational thinking, logical reasoning, and creative expression over syntax or production code generation. Scratch has been used in elementary schools worldwide with over 70 million projects created on the platform. Success is measured by whether students understand loops, conditionals, and events, not by whether they can write production code.

W3Schools targets beginners and professionals seeking to learn or reference web development syntax. The platform serves 70 million monthly visitors ranging from complete beginners taking first steps in HTML to experienced developers looking up JavaScript functions. W3Schools emphasizes easy-to-understand explanations and hands-on practice through the Try It Yourself editor. The platform also offers teacher dashboards and student management tools for classroom use.

Code output goals crystallize these differences. NeoDev generates semantic HTML with verbose, descriptive class names such as navigation-container and hero-section-content that explicitly communicate structural intent and design

patterns, as documented in Chapter 4. The output is unminified and formatted for readability with consistent indentation and optional comments, prioritizing transparency so students can study, understand, and modify the generated code without confusion. Educational clarity is the primary optimization goal. Generated code is typically 15 to 20 percent larger than theoretically optimal because additional naming, spacing, and structure communicate meaning and facilitate learning. Each class name teaches semantic HTML principles, each indentation level shows nesting relationships, and each component structure reinforces web development patterns.

Builder.io generates code optimized for professional production environments with semantic, contextual class names that align to existing design systems and coding conventions. The platform emphasizes clean, contextual class names and CSS variable names aligned to design systems rather than minimal abbreviations. Builder.io's code generation can be customized through Custom Instructions to match specific team coding standards, framework preferences, and project requirements. The output is designed for

deployment into professional codebases where code size, framework compatibility, and integration with existing systems are primary concerns. Optimization strategies can improve user experience metrics like page load time and reduce bandwidth costs at production scale. Developers using Builder.io study and modify the generated code extensively before and after deployment to integrate it with backend logic, state management, and existing architecture, but the tool's purpose is to accelerate the translation from design to deployable code rather than to teach web development fundamentals.

FigmaToCode generates responsive layouts optimized for each target framework with special handling for framework-specific patterns and best practices. The plugin produces clean, maintainable code by using an intermediate representation that allows transformations before generation. The output aims for production quality while maintaining readability. When features are unsupported in the target framework, the plugin provides warnings rather than generating invalid code. The code can be further refined by selecting individual items rather than whole

pages, enabling componentization where developers can replicate elements using loops. The plugin handles color variables for theme-consistent output and processes gradients and effects with specialized conversion logic tailored to each framework. Because the conversion algorithms are open source, developers can study the generated code alongside the transformation logic to understand how design properties map to code constructs.

Webflow generates clean, semantic, and well-structured code that developers can edit in any code editor. The exported HTML follows standard conventions with properly nested elements. CSS is modular and follows consistent naming conventions based on element names assigned in the Designer. JavaScript includes code necessary for interactions and animations created in the Designer. The code is described as production-quality and optimized for performance. Webflow's code serves dual purposes: direct deployment for static sites or as a starting point for developers adding custom functionality. The platform cannot be used for teaching code structure because users design

visually without manipulating code directly during the creation process.

Scratch generates no web code. Programs consist of block stacks that execute in the Scratch runtime environment. The visual representation serves as both the programming interface and the program itself. Students learn programming concepts through block manipulation without seeing textual code. This abstraction is intentional, designed to prevent syntax frustration and enable focus on computational thinking.

W3Schools does not generate code. Users write HTML, CSS, and JavaScript directly in text editors provided by the platform. The Try It Yourself editor executes user-written code and displays results, but the platform provides tutorials and examples rather than code generation. Learning occurs through reading explanations, studying examples, and writing code manually. This direct text-based approach teaches syntax and structure explicitly.

The fundamental difference across platforms is not code quality or naming conventions but rather target audience and

optimization priorities. NeoDev optimizes for students learning web development through explicit, transparent code that demonstrates HTML structure and CSS application. Builder.io optimizes for professional developers who need production-ready code that integrates with existing systems and follows team conventions. FigmaToCode optimizes for developers who need free, transparent, customizable design-to-code conversion without vendor lock-in. Webflow optimizes for designers who want visual control without coding. Scratch optimizes for children learning programming concepts without syntax barriers. W3Schools optimizes for syntax learners who want hands-on practice with direct coding. All platforms generate or support quality output appropriate to their purposes. They differ in what quality means for their respective users. NeoDev emphasizes learning readability. Builder.io emphasizes professional integration aligned to team standards. FigmaToCode emphasizes customizable, inspectable conversion logic. Webflow emphasizes design fidelity. Scratch emphasizes conceptual understanding. W3Schools emphasizes syntax mastery.

System Evaluation Results

Methodology and Scope

This study employs usability and accessibility constructs derived from ISO 9241-110 and WCAG 2.0. Data were collected from non-expert users to assess perceived usability and accessibility rather than formal standards compliance. This approach captures the user's lived experience with the interface, focusing on how intuitive and inclusive the application feels during practical task execution.

ISO 9241: Perceived Usability Analysis

The evaluation utilized a checklist grounded in ISO 9241-110 dialogue principles. Users performed representative tasks—such as navigating the workspace, manipulating blocks, and generating outputs—to determine the system's effectiveness as can be seen in Table 21.

Principle	Perceived Satisfaction Rate
Task Suitability & Consistency	90%
Self-Descriptiveness	65%
Error Tolerance & Guidance	60%
Individualization & Adaptability	30%

Table 21. ISO 9241 Results

The results indicate that the system demonstrates high perceived usability in core areas. 90% of evaluated interactions were perceived as consistent and well-aligned with user workflows, allowing tasks to be completed without external documentation. However, only 60% of users felt the error messaging provided sufficient guidance, and perceived adaptability was the lowest at 30%, suggesting that while the system is structurally sound, it lacks the personalization features expected by modern users.

WCAG 2.0: Perceived Accessibility Analysis

Accessibility was assessed through the four foundational principles: Perceivable, Operable, Understandable, and Robust.

1. Perceivable

Visual clarity was rated highly, with over 85% of interface elements meeting perceived legibility and contrast needs. While the default color palette and icon spacing were found to be intuitive, roughly 15% of decorative elements lacked the descriptive attributes necessary for a fully transparent experience for all user types.

2. Operable

The evaluation revealed a significant distinction between general navigation and workspace interaction:

- General Navigation: Users found that 100% of top-level menus and configuration dialogs were accessible via keyboard.
- Workspace Interaction: Because the application is a visual IDE designed for pointer-based input, keyboard operability within the block-based workspace was perceived at only 20%.

While the system is highly responsive to mouse-driven actions, the complex drag-and-drop mechanics represent a technical delimitation. This results in an overall 40%

perceived operability for users attempting to navigate without a pointing device.

3. Understandable

This was the strongest category, with a 95% satisfaction rate. Users perceived the terminology and block labels as highly consistent. The system’s "guardrail" mechanics, which prevent invalid block connections, successfully reduced user error, making the interface feel safe and predictable for non-experts.

4. Robust

The system’s output and structural integrity were perceived as highly stable. 90% of the generated HTML/CSS code passed structural syntax requirements, ensuring that the user’s work remains functional and consistent across different web browsers.

CHAPTER 5 SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS

Summary of the Proposed Study, Design, and Implementation

Learning web development often presents a steep learning curve due to the distinct coding paradigms and complex syntax of HTML, CSS, and JavaScript. While existing GUI-based site builders abstract this complexity, they frequently stray from foundational concepts, limiting the developer's long-term learning and flexibility. To bridge this gap, this study introduced NeoDev, a visual-focused Software-as-a-Service (SaaS) learning tool designed to simplify web development while preserving its core structural principles.

System Design and Core Features

The NeoDev platform utilizes a block-based, drag-and-drop interface inspired by educational tools like Scratch. The system architecture is divided into three primary modules:

- UI Module: Provides a playground environment where users connect "Tag Blocks" representing HTML elements and "Style Blocks" representing CSS declarations. A Live Website Preview allows users to see real-time updates through browser-based virtualization.
- Application Layer: Contains the core logic for converting visual blocks into an intermediary JSON-based hierarchical tree structure.
- Intelligence Layer: Features a GraphSAGE-based Neural Network Classifier that identifies UI components (e.g., navigation bars, headers) and provides automated insights or "Did You Know?" warnings to guide the user.

Implementation and Technical Framework

The study employed a developmental research design, focusing on two simultaneous paths: system development and model training.

- GraphSAGE Implementation: Unlike traditional GNNs, GraphSAGE was chosen for its inductive learning capabilities, allowing it to generalize to unseen

Document Object Model (DOM) structures. The model was trained on a comprehensive dataset of 6,000 samples, including web-scraped real-world components and programmatically generated synthetic data.

- Code Transpilation: A rules-based system was implemented to transpile the intermediary JSON code into specific front-end framework code, ensuring the output is optimized and natively compatible with the user's chosen framework.
- Data Management: The system utilizes a local file management approach rather than a cloud-based backend, storing user projects as hierarchical JSON structures directly within the local environment to ensure accessibility and performance.

Summary of Findings

Accuracy and Precision of GraphSAGE

The accuracy and precision of the GraphSAGE Classifier was measured using the Macro F1-Score as seen in the Classification Reports. In both the normal and small dataset, GraphSAGE was able to successfully classify all the components with 100% accuracy. However, during Semantic Bleaching testing, GraphSAGE was unable to properly classify headers and footers due to their similar structures. This issue was also seen during inference testing, where the header and footer were incorrectly classified as the NavBar

ISO 9241 and WCAG 2.0 Testing

NeoDev achieved substantial alignment with WCAG 2.0 Level A and partial alignment with Level AA. Strengths were observed in perceivability, understandability, and robustness. The primary area requiring enhancement is operability, particularly the lack of full keyboard access to block manipulation features. Despite this, the system maintains a generally accessible design suitable for a broad range of users.

NeoDev meets several key aspects of ISO 9241 by providing a clear, consistent, and well-structured interface that supports effective task completion and straightforward navigation. Its controls behave predictably, feedback is immediate, and terminology remains uniform across different sections, helping users understand the system without confusion. However, the absence of features such as shortcuts, customizable workflows, and preference retention limits the program's support for efficiency and individual user control. While the system remains usable and coherent, these limitations prevent it from fully addressing ISO 9241's expectations for flexible, user-adaptable interaction.

NeoDev Compared to Builder.io and Figma to Code

NeoDev stands out for its structured and rule-based approach. It relies on user-created JSON inputs, deterministic rule-based transpilation, and an AST generator with a style manager, which collectively ensure predictable, repeatable outputs with no hallucinations. It is optimized for speed and scalability, offering high throughput, low latency, and fast page conversion. This makes NeoDev

well-suited for educational, research, or production environments where consistency, transparency, and performance are critical.

Builder.io, by contrast, focuses on AI/LLM-driven workflows and visual editing. It accepts natural language prompts, Figma designs, and LLM attachments (such as PDFs and images), enabling rapid prototyping and iterative design. Its strengths lie in visual and direct code editing and flexible content creation, but this also implies less deterministic behavior, as outputs depend on LLM interpretation rather than strict rules.

Figma-to-Code tools are the most design-centric. They primarily convert Figma designs directly into code, offering a streamlined path from UI design to production-ready output. However, they lack advanced transpilation logic, structured intermediate representations, and performance guarantees. Their role is focused on design handoff rather than intelligent code generation or optimization.

Conclusions

The researchers would like to conclude the following:

1. GraphSAGE demonstrates strong potential as a classifier for web UI structures, but its current performance is significantly constrained by dataset size and diversity. The model achieved high accuracy and F1-scores across component-only, element-only, and hierarchical classification tasks. However, the presence of near-perfect predictions and inconsistent class performance across all experiments indicates overfitting. This suggests that GraphSAGE successfully learned structural patterns within the dataset but lacked sufficient variation to generalize beyond the training samples. Therefore, while the architecture is appropriate for DOM-based hierarchical classification, its full effectiveness depends on access to larger and more representative datasets.
2. NeoDev successfully integrates a visual block-based editor with AI-driven classification and code transpilation, demonstrating the feasibility of a low-code learning environment for web development. The

system was able to transform user-created visual structures into hierarchical JSON, classify components through GraphSAGE, generate insights, and transpile the optimized structure into front-end framework code. This confirms that a hybrid approach combining block-based learning, graph-based classification, and rule-based transpilation can streamline the web development workflow and support beginner learning.

3. NeoDev aligns with several core usability principles under ISO 9241, but it lacks advanced user-support and personalization features expected of mature interactive systems. The expert evaluation validated the system's clarity, consistency, predictable behavior, and alignment with user expectations. However, areas such as personalization, adaptive guidance, error explanation, and flexible navigation were only partially satisfied. These gaps indicate that while the system is usable, it requires additional development to fully comply with ISO 9241 dialogue principles, especially for novice users.
4. NeoDev satisfies many essential WCAG 2.0 Level A and partial Level AA requirements, particularly in

perceivability, understandability, and robustness. The system demonstrated strong visual clarity, consistent terminology, and stable rendering across browsers. However, full operability, especially keyboard-only navigation for the block-based editor, remains a critical accessibility limitation. This limitation is also common among similar block-based environments but must be addressed for compliance with higher levels of WCAG and broader accessibility adoption.

5. The implemented system demonstrates a functional and technically sound proof-of-concept, but further expansion is required to achieve production-grade reliability and pedagogical maturity. While the integration of GraphSAGE, visual blocks, and rule-based transpilation worked as intended, the system would benefit from enhancements in dataset scale, accessibility compliance, scalability, and instructional features to fully realize its potential as a learning platform.
6. NeoDev effectively bridges the gap between no-code abstraction and traditional syntax-heavy learning. By utilizing a block-based interface that preserves the

fundamental separation of concerns, HTML for structure and CSS for styling, the system successfully fosters design thinking and logical reasoning without the immediate barriers of complex coding paradigms.

Recommendations

The researchers would like to recommend the following:

1. Expand the dataset substantially before retraining GraphSAGE.
2. Augment the dataset by incorporating real-world HTML-CSS structures from diverse sources.
3. Implement regularization strategies beyond hyperparameter tuning to mitigate overfitting.
4. Enhance ISO 9241 compliance by adding user-support and personalization features.
5. Conduct extensive user studies once the platform stabilizes.
6. Integrate more advanced transpilation rules and support additional frameworks.
7. Optimize system scalability and server performance for production use.

References

[1] T. Berners-Lee, Weaving the Web: The Original Design and Ultimate Destiny of the World Wide Web by Its Inventor. Harper San Francisco, 1996.

[2] E. Meyer, CSS: The Definitive Guide. O'Reilly Media, 2018.

[3] J. Duckett, HTML & CSS: Design and Build Websites. John Wiley & Sons, 2011.

[4] S. Krug, Don't Make Me Think, Revisited: A Common-Sense Approach to Web Usability. New Riders, 2014.

[5] R. Feldman, Elm in Action. Manning Publications, 2020.

[6] J. Nielsen, Designing Web Usability: The Practice of Simplicity. New Riders, 2021.

[7] M. Resnick et al., "Scratch: Programming for All," Communications of the ACM, vol. 52, no. 11, pp. 60–67, 2009.

[8] M. A. Kuhail, S. Farooq, R. Hammad, and A. Bahja, "Characterizing Visual Programming Approaches for End-User Developers: A Systematic Review," IEEE Access, vol. 9, pp.

13544-13565, Jan. 2021. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=9320477

[9] B. A. Myers, "Taxonomies of visual programming and program visualization," J. Vis. Lang. Comput., vol. 1, no. 1, pp. 97–123, Mar. 1990.

[10] M. M. Burnett and M. J. Baker, "A classification system for visual programming languages," J. Vis. Lang. Comput., vol. 5, no. 3, pp. 287–300, Sep. 1994.

[11] "MIT Scratch—About," Scratch, 2025. [Online]. Available: https://scratch.mit.edu/about/. [Accessed: 02-Feb-2025].

[12] H. Montiel and M. G. Gomez-Zermeño, "Educational Challenges for Computational Thinking in K–12 Education: A Systematic Literature Review of 'Scratch' as an Innovative Programming Tool," Computers, vol. 10, no. 6, p. 69, May 2021, doi: 10.3390/computers10060069.

[13] E. W. Patton, M. Tissenbaum, and F. Harunani, "MIT App Inventor: Objectives, design, and development," in Springer eBooks, 2019, pp. 31–49. doi: 10.1007/978-981-13-6528-7_3.

[14] S. C. Pokress and J. J. D. Veiga, "MIT App Inventor: Enabling personal mobile computing," arXiv preprint, arXiv:1310.2830, 2013.

[15] G. Inc, "Enterprise LCAP (Low-Code Application Platforms) Reviews 2021 | Gartner Peer Insights," Gartner, 2021. [Online]. Available: https://www.gartner.com/reviews/market/enterprise-low-code-application-platform

[16] A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, "Supporting the understanding and comparison of low-code development platforms," in Proc. 46th Euromicro Conf. Software Eng. and Advanced Applications (SEAA), 2020, pp. 171–178. doi: 10.1109/seaa51224.2020.00036.

[17] Figma, "Figma: The Collaborative Interface Design Tool," Figma, 2024. [Online]. Available: https://www.figma.com/

[18] E. Y. Wijaya, M. Arif, N. Aini, and Y. N. Putri, "UI/UX Web-Based Learning Design with UCD Approach to Basic Programming using FIGMA," bit-Tech, vol. 6, no. 3, pp. 412–420, Jul. 2024, doi: 10.32877/bt.v6i3.1534.

[19] H. Nguyen, "Website design and development," 2020. [Online].Available: https://www.researchgate.net/publication/390486594_Web_Development_and_Design

[20] I. Rubezius, "Why we built Ycode: A personal message from the founder," Ycode.com, Jul. 29, 2020. [Online]. Available: https://www.ycode.com/blog/why-we-built-ycode-a-personal-message-from-the-founder

[21] S. Nicola et al., "How to boost ICT skills in students at higher education? A low-code approach," in EDULEARN Proc., vol. 1, pp. 8167–8176, Jul. 2021, doi: 10.21125/edulearn.2021.1652.

[22] "W3Schools.com," W3Schools, 2025. [Online]. Available: https://www.w3schools.com/about/default.asp

[23] "About | The Odin Project," The Odin Project, 2025. [Online]. Available: https://www.theodinproject.com/about

[24] Upskew Pty. Ltd, "Encode for iOS — Upskew Pty. Ltd.," Upskew.com, 2024. [Online]. Available: https://upskew.com/encode-ios/. [Accessed: 27-Jan-2025].

[25] P. McFedries, "Web Design Playground," Webdesignplayground.io, 2025.

[26] "How Builder Works: A technical overview," Builder.io, 2024. [Online]. Available: https://www.builder.io/c/docs/how-builder-works-technical

[27] Builder.io, "Builder.io - Visual Development for Any Tech Stack," Builder.io, 2024. [Online]. Available: https://www.builder.io/. [Accessed: 28-Jan-2025].

[28] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in Proc. Int. Conf. Learn. Representations (ICLR), 2017.

[29] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2017, pp. 1024–1034.

[30] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in Proc. Int. Conf. Learn. Representations (ICLR), 2019.

[31] R. Ying et al., "Graph convolutional neural networks for web-scale recommender systems," in Proc. ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining (KDD), 2018, pp. 974–983.

[32] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.

[33] C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006.

Appendices

Appendix A: ISO 9241 Checklist

Instructions:
Evaluate each item in the checklist by examining how well the system satisfies the corresponding ISO 9241-110 dialogue principles. For every criterion, assign a rating based on your expert judgment using the provided scale.

Score
Yes | No | Partial | N/A

Severity (Optional)
Low / Medium / High

1. Suitability for the Task

No.	Evaluation Item	Score	Severity
1.1	Features required to complete tasks are easy to find.		
1.2	The system provides the necessary information at the right time.		
1.3	The sequence of steps matches the user's natural workflow.		
1.4	Redundant or unnecessary actions are minimized.		
1.5	Users can accomplish tasks without relying on outside documentation.		

2. Self-Descriptiveness

No.	Evaluation Item	Score	Severity
2.1	The purpose of each screen is immediately understandable.		
2.2	Labels and icons clearly indicate their functions.		
2.3	The system provides helpful cues (tooltips, hints, feedback messages).		
2.4	Navigation elements make the current location obvious.		
2.5	Error messages clearly state what went wrong and how to fix it.		

3. Controllability

No.	Evaluation Item	Score	Severity
3.1	Users can easily undo or redo actions.		

No.	Evaluation Item	Score	Severity
3.2	Users can cancel ongoing operations.		
3.3	Users can freely navigate backward and forward.		
3.4	No unexpected actions occur without confirmation.		
3.5	Inputs and commands produce predictable results.		

4. Conformity with User Expectations

No.	Evaluation Item	Score	Severity
4.1	Similar actions look and behave consistently across the system.		
4.2	Terminology is consistent throughout the design.		
4.3	The system uses familiar patterns and conventions.		
4.4	Layout is consistent across pages or modules.		
4.5	Unexpected behavior or inconsistencies are minimized.		

5. Error Tolerance

No.	Evaluation Item	Score	Severity
5.1	The system prevents common errors before they occur.		
5.2	Input validation catches mistakes early.		
5.3	Error messages are constructive and actionable.		
5.4	Users can recover from errors without losing progress.		
5.5	The system avoids irreversible destructive actions.		

6. Suitability for Individualization

No.	Evaluation Item	Score	Severity
6.1	Users can adjust settings (e.g., theme, layout, preferences).		
6.2	The system supports different skill levels.		
6.3	Users can personalize workflows or shortcuts.		
6.4	The system remembers user preferences across sessions.		
6.5	Display and interaction options are configurable.		

7. Suitability for Learning

No.	Evaluation Item	Score	Severity
7.1	New users can understand how to start tasks without training.		
7.2	The system offers examples, guides, or walkthroughs where needed.		
7.3	Functions are discoverable without trial-and-error frustration.		
7.4	Advanced features become easier to use as skill increases.		

No.	Evaluation Item	Score	Severity
7.5	Navigation paths remain logical and easy to remember.		

Appendix B: WCAG Checklist

WCAG 2.0 FULL CHECKLIST (A, AA, AAA)

Format: ✔ Pass | ✖ Fail | ⚠ Partial | N/A

1. PERCEIVABLE

1.1 Text Alternatives

Score	Success Criterion Number	Level	Requirement	Checklist Item
	1.1.1 Non-text Content	A	All non-text content must have text alternatives.	Images, icons, buttons, CAPTCHA alternatives, charts, etc. have alt text or descriptions.

1.2 Time-based Media

Score	Success Criterion Number	Level	Requirement	Checklist Item
	1.2.1 Audio-only and Video-only (Prerecorded)	A	Provide text alternatives for audio-only and video-only media.	Prerecorded sound has transcript; prerecorded video has description.
	1.2.2 Captions (Prerecorded)	A	Captions for prerecorded video with audio.	Closed/open captions available.
	1.2.3 Audio Description or Media	A	Provide audio description OR text alternative.	An audio-described version or transcript exists.

	Alternative (Prerecorded)			
	1.2.4 Captions (Live)	AA	Captions for live audio in video.	Live events or streams have captions.
	1.2.5 Audio Description (Prerecorded)	AA	Audio description for prerecorded video.	If video has important visual info, audio description is added.
	1.2.6 Sign Language (Prerecorded)	AAA	Sign language interpretation available.	Sign language version provided.
	1.2.7 Extended Audio Description (Prerecorded)	AAA	Extended audio descriptions where pauses are insufficient.	Extended narration provided.
	1.2.8 Media Alternative (Prerecorded)	AAA	Full text alternative for video content.	A complete text script describing all visual + audio.
	1.2.9 Audio-only (Live)	AAA	Live audio has text alternatives.	Live-only audio events have transcripts or notes.

1.3 Adaptable

Score	Success Criterion Number	Level	Requirement	Checklist Item
	1.3.1 Info and Relationships	A	Structure must be conveyed (headings, lists, forms).	Proper HTML semantics reflect visual structure.
	1.3.2 Meaningful Sequence	A	Reading/traversal order is logical.	Keyboard, screen reader order matches visual order.
	1.3.3 Sensory Characteristics	A	Instructions do not rely only on shape, size, location.	"Click the blue button" is avoided; alternatives provided.

	1.3.4 Orientation	AA	Content works in both portrait/landscape.	No forced orientation.
	1.3.5 Identify Input Purpose	AA	Inputs that collect personal data have proper autocomplete.	Autocomplete fields (name, email, address) are tagged correctly.
	1.3.6 Identify Purpose	AAA	Icons, components have standard purposes.	ARIA roles and autocomplete attributes used consistently.

1.4 Distinguishable

Score	Success Criterion Number	Level	Requirement	Checklist Item
	1.4.1 Use of Color	A	Color cannot be the sole means of communication.	Status, errors, links have alternative cues.
	1.4.2 Audio Control	A	Auto-playing audio > 3 seconds must be stoppable.	Users can pause/stop or control volume.
	1.4.3 Contrast (Minimum)	AA	Text contrast ≥ 4.5:1 (normal) or 3:1 (large).	Check all text/background pairs.
	1.4.4 Resize Text	AA	Text can be resized 200% without loss.	Layout remains functional at 200% zoom.
	1.4.5 Images of Text	AA	Avoid images of text unless essential.	Live text used where possible.
	1.4.6 Contrast (Enhanced)	AAA	Text contrast ≥ 7:1 (normal) or 4.5:1 (large).	Enhanced contrast validated.
	1.4.7 Low or No Background Audio	AAA	Audio has no or low background sounds.	Speech-only audio is clean.

	1.4.8 Visual Presentation	**AAA**	**Users can control spacing, colors, and background.**	**No text blocks > 80 characters, sufficient line height.**
	1.4.9 Images of Text (No Exception)	**AAA**	**Images of text are never used.**	**All text rendered as actual text.**

2. OPERABLE

2.1 Keyboard Accessible

Score	Success Criterion Number	Level	Requirement	Checklist Item
	2.1.1 Keyboard	**A**	**All functions accessible via keyboard.**	**No mouse-only features.**
	2.1.2 No Keyboard Trap	**A**	**Keyboard users can freely navigate and exit components.**	**Tab focus never gets stuck.**
	2.1.3 Keyboard (No Exception)	**AAA**	**All functionality is keyboard-operable with no exceptions.**	**Strict keyboard accessibility enforced.**

2.2 Enough Time

Score	Success Criterion Number	Level	Requirement	Checklist Item
	2.2.1 Timing Adjustable	**A**	**Users can adjust, extend, or disable time limits.**	**Sessions, forms, timers adjustable.**
	2.2.2 Pause, Stop, Hide	**A**	**Auto-updating content must be pausable.**	**Slideshows, tickers, animations controllable.**

	2.2.3 No Timing	AAA	No time limits at all.	Application does not restrict user time.
	2.2.4 Interruptions	AAA	Users can postpone interruptions.	System notifications can be disabled.
	2.2.5 Re-authentication	AAA	Data not lost after login timeout.	Work preserved when session refreshes.

2.3 Seizures

Score	Success Criterion Number	Level	Requirement	Checklist Item
	2.3.1 Three Flashes or Below Threshold	A	No content flashes more than 3×/second.	Animations comply.
	2.3.2 Three Flashes	AAA	No flashing at all.	Strict no-flash rule.

2.4 Navigable

Score	Success Criterion Number	Level	Requirement	Checklist Item
	2.4.1 Bypass Blocks	A	Provide skip links to bypass repeated content.	"Skip to main content" link provided.
	2.4.2 Page Titled	A	Descriptive page titles.	Unique, meaningful titles.
	2.4.3 Focus Order	A	Focus order is logical and predictable.	Keyboard navigation flows linearly.
	2.4.4 Link Purpose (In Context)	A	Link purpose clear from context.	No ambiguous "Click here."

	2.4.5 Multiple Ways	**AA**	**Provide more than one way to reach pages.**	**Search, menu, breadcrumb, sitemap.**
	2.4.6 Headings and Labels	**AA**	**Headings/labels are descriptive.**	**H1–H6 convey hierarchy.**
	2.4.7 Focus Visible	**AA**	**Keyboard focus indicator clearly visible.**	**No invisible focus outlines.**
	2.4.8 Location	**AAA**	**Show the user's current location.**	**Breadcrumbs, active menu states.**
	2.4.9 Link Purpose (Link Only)	**AAA**	**Link purpose clear from text alone.**	**Link text self-explanatory.**
	2.4.10 Section Headings	**AAA**	**All content sections have headings.**	**Logical division with headings.**

3. UNDERSTANDABLE

3.1 Readable

Score	Success Criterion Number	Level	Requirement	Checklist Item
	3.1.1 Language of Page	**A**	**Declare language of the page.**	**<html lang="en"> present.**
	3.1.2 Language of Parts	**AA**	**Mark inline changes in language.**	**Foreign phrases marked with lang=.**
	3.1.3 Unusual Words	**AAA**	**Provide definitions.**	**Glossary or help available.**
	3.1.4 Abbreviations	**AAA**	**Expand abbreviations.**	**Acronyms defined on first use.**

	3.1.5 Reading Level	AAA	Lower secondary education reading level.	Simplified text or summaries.
	3.1.6 Pronunciation	AAA	Provide pronunciation guides when needed.	Tooltips or phonetic hints.

3.2 Predictable

Score	Success Criterion Number	Level	Requirement	Checklist Item
	3.2.1 On Focus	A	Focus does NOT trigger unexpected changes.	No auto-submission or navigation on focus.
	3.2.2 On Input	A	Input does NOT cause unexpected changes.	Forms don’t auto-submit without warning.
	3.2.3 Consistent Navigation	AA	Navigation order consistent across pages.	Menus don’t rearrange.
	3.2.4 Consistent Identification	AA	Same elements identified consistently.	Icons/buttons labeled uniformly.
	3.2.5 Change on Request	AAA	Only user-requested changes occur.	No sudden popups or auto-redirects.

3.3 Input Assistance

Score	Success Criterion Number	Level	Requirement	Checklist Item
	3.3.1 Error Identification	A	Input errors identified clearly.	Incorrect fields highlighted with explanations.

	3.3.2 Labels or Instructions	**A**	**Forms have clear labels/instructions.**	**Each field is clearly labeled.**
	3.3.3 Error Suggestion	**AA**	**Suggestions provided for errors.**	**Example: "Use email format name@email.com."**
	3.3.4 Error Prevention (Legal/Financial/Data)	**AA**	**Confirmation + review before submission.**	**Critical transactions require verification.**
	3.3.5 Help	**AAA**	**Context-sensitive help is available.**	**Tooltips or help icons.**
	3.3.6 Error Prevention (All)	**AAA**	**All submissions reviewed before final action.**	**Preview and confirmation for all forms.**

4. ROBUST

4.1 Compatible

Score	Success Criterion Number	Level	Requirement	Checklist Item
	4.1.1 Parsing	**A**	**Valid markup; no broken HTML.**	**Elements nested correctly, no duplicate IDs.**
	4.1.2 Name, Role, Value	**A**	**UI components expose accessible states via ARIA.**	**Buttons, menus, modals have accurate roles.**
	4.1.3 Status Messages	**AA**	**Status messages programmatically indicated.**	**Alerts and updates announced by screen readers.**

Appendix C: Disclaimer

This research project and its corresponding documentation entitled "NeoDev: A Visual Web Dev IDE and Learning Tool utilizing Graph-Based Classification and Front-End Framework Conversion via GraphSAGE and Rules-Based System Code Transpilation" is submitted to the College of Information and Communications Technology, West Visayas State University, in partial fulfillment of the requirements for the degree, Bachelor of Science in Computer Science. It is the product of the authors' work, except where indicated text. Portions of this study were assisted by artificial intelligence tools for grammar, spelling, and language refinement. All ideas, interpretations, analyses, and conclusions are solely those of the authors.

We hereby grant the College of Information and Communications Technology permission to freely use, publish in local or international journal/conferences, reproduce, or distribute publicly the paper and electronic copies of this software project and its corresponding documentation in whole or in part, provided that we are acknowledged.

West Visayas State University

COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY

La Paz, Iloilo City, Philippines

Rei Ebenezer G. Duhina

Marc Joshua H. Escueta

John Albert T. Claveria

Joshua Prinze C. Calibjo

Prince Alexander D. Malatuba